실전반 4회차 — 낭독 대본

[이론 01] 실전반 4회차

[SLIDE 1] 일하는 AI와 업무 지침·도구의 연결

지난 회차까지 판단에 쓸 구조와 지식을 정리했습니다. 이번에는 에이전트가 그 지식과 도구를 어떻게 사용하는지 보겠습니다. 목표에 따라 조회하고 결과를 읽어 다음 행동을 정하는 흐름부터, 그 수행 방법을 스킬로 남기고 도구를 MCP로 연결하는 과정까지 이어갑니다.

[SLIDE 2] 학습목표

LLM과 RAG, 에이전트가 맡는 역할을 구별하겠습니다. 원인을 추론할 때는 가설과 확인된 사실을 나누고, 근거가 없으면 어떤 판단을 보류할지 정합니다. 수행 방법은 SKILL.md에 절차와 예외로 남기겠습니다. 마지막에는 MCP의 구성과 권한을 이해하고, 연결됐다는 사실뿐 아니라 실제 호출의 입력과 결과를 확인하겠습니다.

[SLIDE 3] 목차

에이전트의 원리와 실행 기록을 먼저 배우고, 포털에서 업무 에이전트를 구성한 뒤 원인 추론 기능을 직접 만들겠습니다. 이어서 스킬을 배우고 회의록을 작업 목록으로 바꾸는 지침을 작성합니다. MCP와 연결의 안전 기준을 배운 뒤에는 포털의 업무 도구와 로컬의 파일·Git·시간 서버를 확인하겠습니다. 마지막에 두 묶음의 복습과 용어, 여섯 퀴즈를 함께 정리합니다.

[SLIDE 4] LLM · RAG · 에이전트의 구분

그럼 첫 단원, LLM과 RAG, 에이전트의 구분부터 시작하겠습니다. 쿨러 사건에서 에이전트가 맡았던 일을, 우리가 흔히 쓰는 채팅형 AI에게 그대로 맡긴다고 해 볼까요? 뭐가 모자랄까요? 경보 한 건의 처리를 맡기려면 AI에 무엇이 더 있어야 하는지, 세 방식을 같은 요청으로 견주면서 찾아보겠습니다.

[SLIDE 5] 경보에서 승인까지: 에이전트가 맡은 구간

먼저 쿨러 사건의 시간표에서, 에이전트가 맡았던 구간이 어디인지 볼게요. 그림은 왼쪽에서 오른쪽으로 읽으면 됩니다. 맨 왼쪽 종 모양이 쿨러 성능 저하 경보가 난 순간이고, 맨 오른쪽 사람 그림이 운전원이 승인한 순간이에요. 정확한 시각은 그림마다 붙은 이름표에 적혀 있어요. 둘 사이가 2분이 채 안 됩니다.

아래 시간 축으로 내려가 볼게요. 진하게 칠한 긴 구간이 1분 반쯤 됩니다. 화면에는 약 94초로 적었어요. 이 시간 동안 일한 건 사람이 아니라 가운데 에이전트입니다. 에이전트 그림 둘레에 원호가 감겨 있죠. 같은 일을 한 번이 아니라 여러 번 오가며 했다는 표시예요. 그 결과로 오른쪽의 겹친 카드, 추천안이 나왔습니다. 카드가 겹쳐 있는 건 추천 하나에 대안 셋이 함께 들어 있어서고요.

그 옆에 옅게 칠한 짧은 구간은 십여 초예요. 추천안이 나온 뒤 운전원이 승인하기까지 걸린 시간이죠. 그러니까 이 2분 가운데 대부분은, 에이전트가 판단을 준비하는 데 쓴 시간이었던 겁니다.

이 비율이 말해 주는 게 있어요. 사람이 한 일은 준비된 추천안을 보고 승인할지를 정하는 데 모였습니다. 원인을 찾고 후보를 견주는 긴 준비는 에이전트가 맡았고, 마지막 결정은 사람이 내린 거죠. 이번 회차에 들여다볼 건 바로 그 긴 준비 구간입니다.

[SLIDE 6] 94초 동안 이어진 네 가지 일

그 94초 동안 에이전트는 뭘 했을까요? 화면에 네 칸으로 놓았습니다. 값을 읽고, 원인을 가리고, 후보를 견주고, 추천안을 제출했어요.

첫 칸은 센서 값을 읽는 일이에요. 유온, 냉각 효율, 팬 진동, 캐비닛 온도를 읽었습니다. 경보는 유온이 오른다고 알렸는데, 왜 팬 진동과 캐비닛 온도까지 읽었을까요? 유온만 봐서는 원인을 가를 수가 없거든요. 팬이 제대로 도는지, 주변이 더운지를 따로 봐야 원인 후보를 하나씩 지울 수 있습니다. 둘째 칸에서는 그 값으로 원인을 가렸어요. 후보 셋 가운데 쿨러 핀 오염이 남았죠. 셋째 칸에서는 조치 후보 넷을 견주었습니다. 규정에 걸리는지, 조치하고 나면 유온이 얼마가 될지, 순위는 어떻게 되는지를 따졌고요. 마지막 칸에서 그 결과를 추천안으로 냈습니다.

이 사례에서는 뒤의 일이 앞의 결과를 필요로 합니다. 값을 읽어야 원인을 가릴 수 있고, 원인이 정해져야 어떤 조치를 견줄지가 정해지거든요. 견준 결과가 있어야 제출도 할 수 있고요.

결국 이 1분 반은 질문 하나에 답 하나를 만든 시간이 아닙니다. 앞 단계의 결과를 받아서 다음 단계를 하는, 그런 일이 줄줄이 이어진 시간이에요.

[SLIDE 7] LLM의 응답 생성과 외부 실행의 구분

이 일을 맡길 AI의 출발점은 LLM, 대규모 언어 모델입니다. 앞 회차에서 문서의 지식 후보를 뽑을 때도 썼죠. 우선 도구가 연결되지 않은 모델의 응답 한 번을 보겠습니다.

왼쪽 입력에는 질문과 지시, 참고 자료가 들어갑니다. 모델은 학습한 내용과 지금 받은 자료를 바탕으로 오른쪽 응답을 만들어요. 예를 들어 매뉴얼을 넣고 점검 조건을 정리해 달라고 하면, 문장을 읽고 조건을 정리할 수 있습니다.

그런데 지금 설비의 유온은 어떨까요? 입력에 없고 별도로 조회하지도 않았다면 모델은 현재 측정값을 확인할 수 없습니다. 답에 숫자를 적었다는 것과 센서 값을 읽었다는 것은 다른 일이에요. 화면에 외부 저장소로 가는 화살표가 없는 이유가 이것입니다.

도구를 연결한 채팅 서비스를 써 보셨다면 질문 한 번에 검색까지 해 주는 걸 보셨을 거예요. 그때는 모델만 있는 것이 아니라, 모델의 요청을 받아 검색을 실행하는 프로그램도 함께 동작합니다. 모델은 도구를 부르겠다는 요청을 만들 수 있고, 실제 조회는 연결된 실행 프로그램이 맡아요. 이 둘을 구분해야 뒤에서 에이전트가 한 일을 정확히 읽을 수 있습니다.

[SLIDE 8] LLM만으로는 채울 수 없는 세 자리

예를 하나 들어 볼게요. LLM에게 HYD-01의 유온이 지금 몇 도냐고 물으면 어떻게 될까요? 그 답은 시계열 DB에 있습니다. 모델이 학습한 내용 안에는 없죠. 이렇게 LLM만으로는 채울 수 없는 자리가 화면에 세 개 있어요.

첫째 카드는 지금의 값입니다. 유온, 냉각 효율, 운전 모드는 계속 바뀌어요. 학습이 끝난 뒤에 생긴 값이 모델 안에 있을 수는 없겠죠.

둘째 카드는 이 공장의 문서예요. 모델이 그 문서를 학습했는지, 최신 승인본의 조건까지 알고 있는지 우리는 보장할 수 없습니다. 일반적인 유압 설비 설명이 맞더라도 이 공장의 현재 기준과 같다는 뜻은 아니죠. 그래서 승인된 매뉴얼을 따로 제공하거나 찾아 읽게 해야 합니다.

셋째 카드가 수행이에요. 조회하고, 계산하고, 제출하는 일은 글을 만드는 일과 다릅니다. 모델이 「재고를 조회해 보겠습니다」라고 쓸 수는 있어요. 그렇다고 실제로 조회가 일어난 건 아니죠.

정리하면 값, 문서, 수행. 이 세 자리가 비어 있는 겁니다. 과정 처음에 AI에게 그대로 물으면 그럴듯한 답은 나오지만 그 근거를 확인할 방법이 없다고 했죠. 그 문제를 자리로 나눠 보면 바로 이 셋이에요. 지금의 값을 보지 못하고, 이 설비의 승인된 문서를 확인하지 못하고, 직접 확인해 볼 손발이 없는 겁니다.

[SLIDE 9] RAG의 근거 검색

이 가운데 문서의 빈자리를 메우는 방식이 RAG, 검색 증강 생성입니다. 모델이 배우지 않은 문서로도 답하게 해 주죠. 방법은 간단해요. 질문과 가까운 문서 조각을 먼저 찾고, 그 조각을 질문과 함께 모델에 넣는 겁니다.

화면의 네 칸을 왼쪽부터 볼게요. 먼저 사람이 질문을 합니다. 다음 칸이 검색이에요. 질문과 뜻이 가까운 문서 조각을 찾죠. 셋째 칸, 보강에서는 찾은 조각을 질문과 함께 모델에 넣습니다. 마지막 칸에서 모델이 그 조각을 읽고 글로 답하고요.

여기서 중요한 건 가운데 두 칸이에요. 모델을 다시 학습시키는 게 아닙니다. 답할 때 필요한 문서를 옆에 펼쳐 주는 거죠. 그래서 학습하지 않은 문서의 내용으로도 답할 수 있게 됩니다.

지난 회차에서 지식 그래프와 RAG를 함께 쓸 수도 있다고 했죠. RAG의 핵심은 필요한 자료를 검색해서 답의 근거로 제공하는 데 있습니다. 화면은 문서 조각을 찾는 예지만, 그래프나 데이터베이스에서 조회한 자료도 활용할 수 있어요. 문서 검색은 RAG의 한 구성이지, RAG가 다룰 수 있는 자료의 전부는 아닙니다.

이 그림의 마지막은 검색한 자료로 답을 만드는 단계입니다. 쿨러 경보의 처리를 맡기려면 여기서 더 정해야 할 것이 있어요. 어떤 값을 조회할지, 받은 결과에 따라 무엇을 더 확인할지, 언제 제출하거나 멈출지입니다. 에이전트는 이런 다음 행동을 정하는 구조이고, 그 안에서 자료를 찾을 때 RAG를 쓸 수도 있습니다.

[SLIDE 10] 에이전트의 목표 지향 작업 수행

그 수행의 빈자리까지 메우는 게 에이전트입니다. 조회하고, 계산하고, 제출하는 일까지 맡기려면 어떻게 해야 할까요? 모델이 도구를 부르고, 그 결과를 보면서 다음 행동을 정하게 해야 합니다.

맨 위 줄의 정의부터 볼게요. 이 수업의 에이전트는 목표를 받고, 도구의 결과를 보며 다음 행동을 정하는 시스템입니다. 필요한 일을 이어 가되, 완료 조건이나 멈춤 조건에 이르면 끝냅니다. 결과는 근거가 붙은 답변일 수도 있고, 다음 업무로 넘길 추천안일 수도 있어요.

예를 들어 추천안을 제출하라는 목표라면, 추천 문장을 썼다는 것만으로는 끝나지 않습니다. 필요한 근거를 확인했고, 정해진 항목을 채웠고, 제출 도구가 접수를 확인했는지를 봐야 해요. 반대로 필수 근거가 없거나 권한 밖의 일이 필요하면 완료하지 못한 이유를 남기고 멈춥니다. 스스로 끝났다고 말하는 것과 실제 완료 조건을 만족하는 것을 구분하는 겁니다.

가운데 줄이 도구예요. 모델이 부를 수 있도록 미리 정해 둔 기능입니다. 이름과 입력이 정해져 있어서, 모델은 어느 도구에 뭘 넣을지만 적으면 돼요. 이게 왜 중요할까요? 모델이 아무렇게나 적은 글은 시스템이 받아서 실행할 수가 없거든요. 실행 프로그램은 정해진 이름과 입력 형식, 허용된 권한을 확인해 그 요청을 실제 도구에 전달합니다. 쿨러 사건에서는 원인 진단, 생산오더 조회, 판단 제출 같은 도구가 쓰였죠.

맨 아래 줄이 쿨러 사건의 실제 에이전트입니다. 이름이 냉각 긴급 대응 에이전트예요. 경보 한 건을 받아서 원인을 가리고 추천안을 제출했습니다. 앞에서 본 1분 반 남짓이 바로 이 에이전트가 일한 시간이었고요.

[SLIDE 11] 연결한 기능이 다른 세 구성의 비교

이번에는 연결한 기능이 다른 세 구성을 같은 요청으로 비교해 볼게요. HYD-01의 쿨러 성능 저하 경보에 조치를 정하라는 요청입니다. 표의 오른쪽 열은 아직 보지 말고, 각 방식이 무엇을 돌려줄지 먼저 생각해 보세요.

첫 줄, LLM은 학습한 내용을 근거로 이어질 글을 만듭니다. 그러니 유압 설비에 대한 일반적인 설명이 돌아오죠. 쿨러 성능이 떨어지면 이런 걸 점검한다, 이런 식의 글이요.

둘째 줄, RAG는 찾아온 문서 조각을 근거로 답해요. 매뉴얼을 찾았다면 그 내용을 인용한 설명이 돌아옵니다. 이 공장의 문서를 반영하니까 한 걸음 나아간 거죠. 이 예에서는 문서만 제공했으므로 지금 유온은 확인하지 못합니다. 현재 값을 조회해 제공하는 구성이라면 이 부분은 달라질 수 있어요.

셋째 줄, 에이전트는 도구가 돌려준 결과를 근거로 삼아요. 지금의 값을 읽고, 원인을 가리고, 조치 후보 넷을 견주어서 제출합니다. 그래서 이 설비의 값으로 만든 추천안이 돌아오죠.

그럼 위의 두 줄은 쓸모가 없을까요? 그렇지는 않아요. 쿨러가 어떤 원리로 식히는지, 매뉴얼에 무슨 절차가 있는지 이해하는 데는 도움이 됩니다. 다만 운전원이 지금 이 경보에 승인 버튼을 누를

지 정하는 재료로는 모자라요. 그러려면 이 설비의 지금 값과, 그 값에서 나온 근거가 붙은 결과가 있어야 하니까요.

이 표는 LLM, RAG, 에이전트를 서로 대체하는 세 세대로 나눈 것이 아닙니다. 모델은 응답을 만들고, RAG는 검색한 근거를 제공하고, 에이전트는 결과를 보며 다음 행동을 정합니다. 함께 쓰일 수 있는 요소들이죠. 우리 업무에 무엇이 필요한지는 답의 길이가 아니라, 확인할 근거와 수행할 일로 정하면 됩니다.

[SLIDE 12] 에이전트의 세 요소: 모델 · 도구 · 실행 고리

그렇게 찾고 계산하고 제출하는 에이전트는, 무엇으로 이루어져 있을까요? 이 수업에서 보는 LLM 기반 에이전트는 크게 요소가 셋이에요. 모델, 도구, 실행 고리입니다. 여기서 눈여겨볼 게 있어요. 모델은 앞에서 본 LLM 그대로라는 겁니다.

화면 왼쪽 말풍선이 모델이에요. 글을 읽고 다음 행동을 정합니다. 가운데가 도구고요. 지식 그래프와 시계열 DB, 표를 실제로 조회하고 계산해서 결과를 돌려주죠. 그다음 원호와 스패너가 실행 고리입니다. 모델과 도구 사이를, 도구 결과에 따라 다음 호출이나 종료를 정하도록 이어 주는 틀이에요.

이 셋을 더한 게 등호 오른쪽의 에이전트입니다. 모델이 더 똑똑해진 게 아니에요. 모델 둘레에 도구와 고리가 붙은 거죠.

그래서 에이전트가 일을 잘하는지 따질 때 모델만 봐서는 안 됩니다. 어떤 도구를 쓸 수 있는지, 고리가 어떻게 돌아가는지가 결과를 함께 정하거든요.

[SLIDE 13] 에이전트를 정의하는 항목: 목표 · 도구 · 업무 지침

요소는 셋으로 같은데, 에이전트 하나하나는 무엇으로 정해질까요? 화면에 두 에이전트의 정의를 나란히 놓았습니다. 왼쪽은 쿨러 사건을 맡은 냉각 긴급 대응 에이전트, 오른쪽은 예비품 재고 경보를 맡는 구매 에이전트예요.

첫 줄은 목표입니다. 무엇을 이뤄야 일이 끝나는지를 정하죠. 왼쪽은 쿨러 경보의 원인을 진단하고, 냉각 조치 후보를 견주어 추천안을 올리는 게 목표예요. 오른쪽은 재고가 기준을 벗어났을 때 필요한 양을 정하고, 공급사를 금액과 납기와 품질로 견주어 발주 추천안을 올립니다.

목표 칸의 화살표 끝을 견주어 보세요. 둘 다 마지막에 올릴 것으로 끝나죠. 왼쪽은 추천안, 오른쪽은 발주 추천안이요. 목표에 마지막에 무엇을 내야 하는지가 들어 있어야, 에이전트가 언제 일이 끝났는지를 알 수 있습니다.

둘째 줄은 업무에서 쓰는 도구예요. 왼쪽은 설비의 지식과 현재 상태, 조치 평가가 필요하고, 오른쪽은 재고와 공급사의 조건을 비교하는 조회가 필요합니다. 두 에이전트가 같은 서버의 도구를 공유하더라도 실제 업무에서 필요한 기능은 다를 수 있어요. 이 표는 그 쓰임을 비교한 것입니다. 개별 호출의 허용 여부는 등록된 권한 목록을 따로 확인해야 해요.

셋째 줄은 업무 지침입니다. 어떤 순서로 일할지를 적어 둔 거고, 에이전트 스킬이라는 파일로 줘요. 파일이 어떻게 생겼는지는 따로 다룰 테니까, 지금은 순서를 정해 주는 지침이라는 것만 기억해 두면 됩니다.

목표와 도구와 지침이 다르면, 같은 모델이라도 전혀 다른 일을 하는 에이전트가 돼요. 실습에서 에이전트를 직접 만들 때도 바로 이 세 가지를 정하게 됩니다.

[SLIDE 14] 도구 호출 한 건: 모델이 적는 것과 도구가 돌려주는 것

에이전트가 도구를 부른다는 게 실제로는 어떤 모습일까요? 쿨러 사건의 처리 기록에서 도구 호출 한 건을 꺼내 왔습니다. 경보가 나고 10초쯤 뒤에 원인 진단 도구를 부른 장면이에요.

화면 왼쪽 맨 윗줄에 도구 이름이 있죠. 원인 진단, 하는 일이 정해진 기능입니다. 그 아래 물음은 모델이 적은 입력이에요. 어느 설비인지, 어떤 종류의 경보인지, 경보 번호가 뭔지, 이 세 가지를 넣었습니다.

물음에 든 게 이 세 값뿐이라는 점도 보세요. 지식 그래프에서 무엇을 찾을지, 어떤 조회문으로 값을 읽을지는 모델이 적지 않았습니다. 그건 도구 쪽에 이미 정해져 있어요. 모델이 할 일은 무엇에 대해 물을지를 정확히 적는 것까지인 거죠.

맨 아래 받음은 도구가 돌려준 결과예요. 1만 자가 넘는 긴 JSON입니다. 이 결과를 만들려고 지식 그래프를 읽고 시계열 DB를 조회한 건 모델이 아니라 도구고요. 맨 윗줄 괄호에는 이 호출 한 번에 걸린 시간이 적혀 있어요. 0.2초 정도입니다.

모델이 만든 것은 도구 이름과 입력을 담은 호출 요청입니다. 실행 프로그램이 그 요청을 도구에 전달하고, 도구는 조회 결과를 돌려줍니다. 그러니 진행 설명에 조회하겠다고 적힌 것만 보지 말고, 실제 호출과 돌아온 결과까지 봐야 수행 여부를 알 수 있어요.

이쯤 되면 단원 처음의 물음에 답할 수 있어요. 경보 한 건을 맡기려면 AI에 무엇이 더 있어야 할까요? 모델은 그대로 둡니다. 거기에 모델이 적은 요청을 받아 실제로 조회하고 계산할 도구, 그리고 그 일을 목표까지 이어 갈 틀을 붙이는 거예요.

[SLIDE 15] 에이전트의 실행 고리

방금 모델 둘레에 붙이는 것 가운데, 일을 목표까지 이어 가는 틀이 있었죠. 그 틀을 들여다보는 게 두 번째 단원, 에이전트의 실행 고리입니다. 경보를 받은 에이전트는 그 1분 반 동안 어떤 순서로 일했을까요? 그리고 왜 도구를 한 번 부르는 걸로 끝나지 않고, 고리처럼 이어져야 했을까요? 쿨러 사건의 처리 기록을 요청 해석부터 종료까지 따라가면서 답을 찾아보겠습니다.

[SLIDE 16] 결과를 확인한 뒤 다음 입력을 정하는 까닭

원인 진단을 부르면 결과를 받을 수 있습니다. 그런데 조치 후보를 평가하려면, 그 결과에 나온 원인을 다음 도구의 입력으로 넣어야 해요. 아직 원인을 읽지 않았다면 다음 입력도 정할 수 없겠죠. 중요한 것은 사용자에게 답을 몇 번 보여 주느냐가 아니라, 필요한 결과를 실제로 확인했느냐입니다.

화면 왼쪽은 값을 확인하지 않고 결론부터 만드는 경우예요. 문장은 매끄러울 수 있지만, 어떤 원인을 근거로 조치를 골랐는지 확인할 수 없습니다. 오른쪽은 진단 결과를 받아 원인을 확인하고, 그 원인에 맞는 후보를 평가합니다. 중간에 필요한 결과를 얻지 못하면 보완하거나 멈춰야 하고요.

이런 순서를 모두 에이전트가 정해야 하는 것은 아닙니다. 진단 다음에 평가를 부르는 절차가 늘 같다면, 프로그램에 그 순서를 고정해 둘 수도 있어요. 이것을 워크플로라고 부릅니다. 에이전트는 결과를 보고 다음에 어떤 기능을 쓸지 정할 필요가 있을 때 도움이 됩니다. 정해진 흐름 안의 일부 일만 에이전트에게 맡길 수도 있죠.

그래서 모든 업무를 에이전트로 바꿀 필요는 없습니다. 고정된 절차로 충분한 부분은 그대로 두고, 상황에 따라 선택해야 하는 부분에 판단의 여지를 주면 돼요. 쿨러 사례에서는 업무 지침이 기본 순서를 주고, 에이전트가 호출 결과를 확인하며 일을 이어 갑니다. 그 구조를 다음 그림에서 펼쳐 보겠습니다.

[SLIDE 17] 실행 고리: 요청 해석에서 종료까지

화면의 다섯 원은 그 실행 고리를 단계로 펼친 거예요. 에이전트를 설명할 때 널리 쓰이는 틀이고, 영어로는 Agent Loop라고 부릅니다. 원마다 그림이 하나씩 들어 있어요. 문서, 목록, 스패너, 돋보기, 그리고 확인 표시가 붙은 문서. 그림을 단계 이름과 짝지어 보면서 왼쪽부터 하나씩 볼게요.

첫째는 요청 해석입니다. 무엇을 받았고 무엇을 내야 하는지 파악하는 단계예요. 둘째는 계획이고요. 목표를 이루려면 어떤 순서로 할지 정합니다. 셋째는 도구 선택과 호출이에요. 어느 도구에 무엇을 물을지 정해서 부르죠. 넷째는 결과 관찰입니다. 받은 결과를 보고 목표에 이르렀는지 판단해요.

목표에 이르렀으면 마지막 종료로 갑니다. 산출물을 제출하고 끝내는 단계죠. 그런데 아직 못 미쳤거나 호출이 뜻대로 안 됐다면 어떻게 할까요? 계속 진행할 근거와 권한이 있다면 위쪽 점선을 따라 계획으로 되돌아갑니다. 계획을 고쳐 다음 호출을 하거나 다시 시도합니다. 필수 근거가 없거나 정한 한도를 넘었다면, 목표를 이루지 못했더라도 이유를 남기고 멈춰야 해요.

다섯 단계 가운데 고리를 고리답게 만드는 건 넷째, 돋보기가 그려진 결과 관찰이에요. 관찰이 없다면 에이전트는 처음 세운 계획을 끝까지 그냥 밀고 가겠죠. 결과가 비었든 틀렸든 상관없이요. 관찰이 있으니까 자기가 한 일의 결과를 보고 다음을 고칠 수 있는 겁니다.

쿨러 사건의 에이전트도 이 다섯 단계를 그대로 밟았어요. 원마다 붙은 이름표의 둘째 줄이 쿨러 사건에서 그 단계에 해당하는 것입니다. 요청 해석은 업무와 입력과 결과 형식을 받는 일이고, 계

획은 지침의 다섯 단계, 호출은 원인 진단과 대안 평가, 관찰은 원인 1순위와 결과의 크기, 종료는 판단 제출과 결과 파일이죠.

[SLIDE 18] 실제 기록 ①: 요청 해석

고리의 첫 단계, 요청 해석부터 실제 기록에서 확인해 보죠. 화면 왼쪽은 처리 기록의 앞부분입니다. 첫 줄에서 경보가 나고 처리 건이 열렸어요. 그리고 1초 뒤, 둘째 줄에서 에이전트 단계가 시작됩니다. 둘째 줄은 이 에이전트에게 배정된 스킬이 하나라는 기록이에요. 앞에서 에이전트를 정의하는 항목에 업무 지침이 있었죠. 그 지침이 실제로 여기서 일에 붙은 겁니다.

에이전트는 시작하면서 몇 가지를 받아요. 오른쪽 첫 설명이 그거예요. 맡은 업무와 지시사항, 입력, 작업 공간, 그리고 결과를 낼 형식입니다. 입력에는 설비 이름과 경보 번호가 들어 있고요. 작업 공간은 이 실행에만 쓰는 전용 폴더예요. 에이전트가 만드는 파일은 이 안에만 쓰고, 끝에 결과 파일을 남기는 곳도 여기입니다. 결과 형식은 무엇을 어떤 항목으로 내야 하는지를 정한 거죠.

결과 형식을 처음부터 받는다는 게 중요해요. 끝에서 무엇을 내야 하는지 알아야, 거기에 이르는 길을 거꾸로 세울 수 있거든요. 원인을 적어 내야 한다면 원인을 가리는 일이 필요하고, 추천과 대안을 내야 한다면 후보를 견주는 일이 필요하겠죠.

기록을 보면, 에이전트가 가장 먼저 한 일은 업무 지침을 읽는 거였어요. 셋째 줄의 스킬 읽음이 그 장면입니다. 냉각 긴급 대응용 에이전트 스킬을 읽고, 무엇을 어떤 순서로 할지 파악한 거죠.

맨 아래 줄은 에이전트가 직접 적은 진행 설명이에요. 먼저 스킬 절차를 읽고, 필요한 판단 도구를 불러오겠다고 적었습니다. 에이전트는 호출 사이사이에 이렇게, 지금 뭘 하려는지를 말로 남겨요.

이 진행 설명은 처리 기록에 그대로 남습니다. 나중에 사람이 기록을 읽을 때, 에이전트가 왜 그 도구를 불렀는지 알 수 있는 단서가 되죠.

진행 설명은 기록을 읽는 데 도움을 주지만, 모델 내부의 생각을 그대로 보여 주거나 실행 성공을 증명하는 것은 아닙니다. 실제로 지침을 읽었는지, 도구가 결과를 돌려줬는지는 해당 호출 기록과 함께 확인합니다.

[SLIDE 19] 계획: 목표를 도구 호출의 순서로 푼 것

고리의 둘째 단계는 계획입니다. 원인을 가리고 조치를 추천하라, 이 목표는 그대로는 실행할 수가 없죠. 어느 도구를 어떤 순서로 부를지로 풀어야 합니다. 그렇게 풀어 놓은 게 계획이에요.

쿨러 사건에서는 업무 지침에 다섯 단계가 적혀 있었습니다. 화면의 다섯 칸이에요. 첫째, 입력에서 설비와 경보의 종류, 경보 번호를 읽습니다. 둘째, 원인 진단을 불러서 원인 1순위와 고장 유형을 얻고요. 셋째, 업무 DB에서 납기 오더와 출하 대기 로트를 확인해요. 넷째, 대안 평가로 후보마다 규정 판정과 예측 유온, 순위를 봅니다. 다섯째, 판단을 제출하고 판단 번호를 받죠.

에이전트는 이 다섯 단계를 뼈대로 삼았습니다.

화면에서 넷째 칸, 대안 평가를 강조했죠. 규정 판정, 예측 유온, 순위. 어디서 본 말들이죠? 조치 절차와 판단 규칙을 다룰 때 결정표로 제외와 감점을 가렸고, 조치하면 유온이 어떻게 될지 미리 계산했고, 성과 지표로 순위를 매겼습니다. 에이전트는 그 판단을 새로 해내는 게 아니에요. 이미 만들어 둔 규칙과 계산을 도구로 불러서 결과를 받는 겁니다.

그런데 하나 기억해 둘 게 있어요. 지침은 어디까지나 뼈대입니다. 실제로 어떤 값을 넣을지, 돌아온 결과를 어떻게 읽을지는 그때그때 에이전트가 정해요.

[SLIDE 20] 실제 기록 ②: 관찰한 결과가 다음 호출의 입력이 된다

돌아온 결과를 읽고 다음에 넣을 값을 정하는 모습, 이건 고리의 셋째와 넷째 단계인 도구 호출과 결과 관찰에서 보입니다. 화면 위쪽은 원인 진단이 돌려준 결과의 요지예요. 긴 결과 가운데 이 장에 필요한 두 줄만 옮겨 왔습니다. 아래쪽은 5초 뒤에 부른 대안 평가의 물음이고요.

위쪽 결과에서 에이전트가 읽은 건 두 가지입니다. 원인 1순위는 쿨러 핀 오염, 고장 유형은 쿨러 냉각 성능 상실이에요. 둘 다 지식 그래프의 식별자로 적혀 있죠. 오른쪽 마지막 설명처럼, 앞머리를 보면 원인인지 고장 유형인지가 나뉩니다.

이제 아래쪽 물음을 보세요. 위쪽 원인 1순위 줄의 값을 눈으로 따라 내려가면, 아래 물음의 원인 칸에 똑같이 들어가 있죠? 고장 유형도 마찬가지예요. 에이전트가 원인을 자기 말로 바꿔 적지 않고, 받은 값을 그대로 넘긴 겁니다. 진단에서 관찰한 결과가 다음 호출의 입력이 된 거예요. 원인을 모르면 후보를 평가할 수 없다고 했죠. 바로 그 연결이 여기서 보이는 거예요.

이 두 호출을 이어 주는 건 모델의 기억이 아니에요. 앞 호출이 돌려준 값입니다. 그래서 대안 평가가 어떤 원인을 바탕으로 했는지가 기록에 그대로 남아요.

그런데 비어 보이는 게 하나 있죠. 앞의 계획에서 셋째 단계는 업무 DB에서 납기 오더와 출하 대기 로트를 확인하는 거였는데, 이 기록에는 진단 다음에 바로 대안 평가가 옵니다. 오른쪽 넷째 설명이 그 답이에요. 이 실행은 업무 값 조회를 따로 부르지 않았습니다. 대안 평가 도구가 평가에 필요한 납기 같은 업무 값을 평가 안에서 직접 읽어 넣었거든요. 단계가 빠진 게 아니라, 다른 도구 안에서 채워진 거죠.

[SLIDE 21] 실제 기록 ③: 큰 결과를 읽는 방법을 바꾸려는 시도

고리가 늘 이렇게 한 번에 이어지지는 않습니다. 이번에는 받은 결과 때문에 방법을 바꾸겠다고 한 장면이에요. 대안 평가가 돌려준 결과가 너무 길었거든요. 12만 자가 넘었습니다. 앞에서 본 원인 진단의 결과보다 열 배 가까이 긴 거죠.

왼쪽 칸을 보세요. 에이전트가 도구의 결과를 한 번에 받을 수 있는 크기에는 한도가 있어요. 이 결과는 그 한도를 넘어서, 본문이 파일로 따로 빠졌습니다. 에이전트가 그 본문을 그대로 받지 못한 거죠. 에이전트는 이 상황을 보고 계획을 바꿉니다. 가운데 칸에서 순위와 규정 판정만 간추려 보겠다고 적었죠. 그리고 6초 뒤, 오른쪽 칸에서는 필요한 항목만 좁혀서 다시 뽑겠다고 했습니다. 따옴표 안은 에이전트가 기록에 남긴 말 그대로예요.

왜 하필 순위와 규정 판정이었을까요? 추천안에 들어갈 게 바로 그거거든요. 어느 후보가 몇 순위인지, 규정에 걸려 빠지거나 감점된 후보는 없는지. 받지 못한 결과를 통째로 다시 달라고 한 게 아니라, 목표에서 거꾸로 따져 꼭 필요한 것만 추린 겁니다.

이 화면으로 확인할 수 있는 것은 결과가 너무 컸다는 상황과, 필요한 항목을 좁혀 보겠다는 진행 설명입니다. 실제로 무엇을 다시 읽었고 필요한 내용을 모두 얻었는지는 이어지는 호출과 결과를 더 봐야 해요. 특히 짧게 만들려고 조치 제한이나 경고를 빼면 안 됩니다. 목표에 필요한 근거를 보존하면서 다룰 수 있는 크기로 읽어야 하는 거죠.

[SLIDE 22] 실제 기록 ④: 종료

고리의 마지막은 종료입니다. 에이전트 단계는 답 문장이 아니라, 추천안을 제출하는 걸로 끝났어요.

맨 윗줄이 판단 제출입니다. 추천안을 제출하는 도구를 불렀고, 판단 번호를 돌려받았죠. 시작한 지 45초쯤 됐을 때예요. 이 번호는 뒤에 만들어지는 작업지시에도 그대로 기록됩니다. 어떤 판단에서 나온 작업인지를 거슬러 찾을 수 있게 하는 거예요. 나중에 정비 기록을 보다가 이 작업이 왜 생겼는지 궁금해지면, 번호 하나로 어느 경보에서 어떤 근거로 나온 판단인지까지 따라갈 수 있습니다.

그다음 에이전트는 제출이 끝났다고 적고, 정해진 형식의 결과 파일을 썼습니다. 처음에 받은 작업 공간에 남기는 파일이죠. 맨 아래 두 줄은 기록의 원문이 아니라, 구간을 적어 둔 주석이에요. 여기서 에이전트 단계가 끝났고, 시작부터 1분 반 남짓 걸렸습니다.

제출 도구의 응답을 받은 시점과 에이전트 단계가 끝난 시점 사이에는 간격이 있습니다. 다만 이 기록만으로 그 시간을 전부 파일 작성에 썼다고 말할 수는 없어요. 이 장에서 확인할 것은 제출 접수와 최종 결과 저장이 서로 다른 완료 항목이라는 점입니다.

이렇게 놓고 보면 그 1분 반 안의 순서가 다 보이죠. 업무 지침을 읽어 요청을 해석하고, 다섯 단계로 계획을 세우고, 진단과 평가를 부르고, 큰 결과를 만났을 때 읽는 방법을 바꾸겠다고 설명하고, 제출과 결과 저장으로 끝냈습니다.

[SLIDE 23] 다른 실행의 기록: 같은 고리, 다른 길

그럼 같은 경보를 다른 실행에서 처리했을 때도 똑같이 일했을까요? 화면은 같은 쿨러 경보를 다른 실행에서 처리한 기록이에요. 이번에는 도구를 부른 순서를 처음부터 끝까지 놓았습니다. 같은 지침을 받았는데도, 앞의 기록과 다른 데가 있어요.

첫 줄은 스킬 파일을 읽는 요청 해석이에요. 둘째 줄부터 볼게요. 에이전트는 일을 시작하자마자 쓸 도구의 이름을 불러왔습니다. 연결된 도구는 더 많았지만, 이번 일에 필요한 것만 골랐어요. 실행 고리의 셋째 단계 가운데 도구 선택이, 이 기록에는 한 줄로 따로 남아 있는 거죠.

넷째 줄에서는 앞에서와 같은 일이 생겼습니다. 대안 평가의 결과가 너무 커서 받지 못했어요. 그런데 이 실행은 다른 길을 택했습니다. 다섯째 줄에서 생산오더와 출하 대기 로트를 읽은 다음, 여

섯째 줄에서 조치 효과 예측을 세 번 불렀어요. 팬만 올렸을 때, 팬을 올리고 부하를 낮췄을 때, 부하만 낮췄을 때 15분 뒤 유온이 어떻게 될지를 하나씩 물은 거죠. 이건 지침의 다섯 단계에는 없던 호출입니다. 평가 결과를 못 받았으니까, 대안의 예상 유온을 다른 도구로 보완해 확인한 거예요. 순서도 하나 달라졌어요. 지침에서는 셋째였던 업무 값 확인이, 이 기록에서는 대안 평가 뒤인 다섯째 줄에 와 있죠.

마지막 메시지도 눈여겨볼 만합니다. 에이전트는 평가의 상세 결과를 열어 보지 못했다는 사실, 그리고 대신 무엇을 근거로 이유를 적었는지를 함께 남겼어요. 앞에서 지침은 뼈대라고 했죠. 어떤 도구를 어떤 차례로 부를지는 결과를 보며 에이전트가 정한다는 것, 이 기록이 바로 그걸 보여 줍니다.

이제 이 단원의 물음에 답할 수 있어요. 도구가 있는데 왜 한 번 부르고 끝내지 못할까요? 앞 호출의 결과를 보고서야 다음 호출의 입력을 적을 수 있고, 결과가 뜻대로 오지 않으면 방법을 바꿔야 하니까요. 그래서 일은 고리로 이어질 수밖에 없고, 같은 지침을 받아도 실행마다 고리를 도는 길이 달라질 수 있는 겁니다.

다만 다른 길을 택했다는 사실이 그 길의 타당성까지 보장하지는 않습니다. 유온 예측은 조치 효과를 비교하는 근거예요. 그 결과만으로 규정 위반이나 작업 제한까지 확인했다고 할 수는 없죠. 필수 판정이 빠졌다면 원래 결과를 확인하거나 보류해야 합니다. 실제 실행 기록은 잘된 점과 부족한 점을 함께 점검하는 자료로 읽어야 해요.

[SLIDE 24] 단순 답변과 업무 산출물

고리의 끝에서 에이전트가 낸 건 답 문장이 아니라 추천안이었죠. 그 차이를 다루는 게 세 번째 단원, 단순 답변과 업무 산출물입니다. 같은 에이전트라도 질문을 받으면 답을 냅니다. 답과 추천안은 뭐가 다를까요? 그리고 추천안은 제출된 뒤에 어디로 넘어갈까요? 이 물음에 답하면서, 질문하는 AI와 일하는 AI의 차이를 마무리해 보겠습니다.

[SLIDE 25] 답변과 업무 산출물의 차이

이 포털에서 에이전트를 쓰는 두 가지 경우를 비교해 보겠습니다. 화면 왼쪽이 단순 답변, 오른쪽이 업무 산출물이에요. 받는 쪽, 형식, 다음 행동, 이 세 줄로 견주어 보겠습니다.

왼쪽부터요. 단순 답변을 받는 쪽은 질문한 사람입니다. 형식은 답과 근거예요. 답할 수 없을 때는 답할 수 없다는 사실과 그 이유고요. 이 질문 기능은 답을 돌려주는 데서 처리가 끝납니다. 사람이 그 답으로 후속 일을 할 수는 있지만, 이 기능이 설비 조치까지 자동으로 이어 주지는 않아요.

오른쪽 업무 산출물은 받는 쪽부터 달라요. 승인할 담당자, 그리고 다음 단계의 시스템이 받습니다. 형식은 원인, 추천, 뒤진 대안, 설명, 판단 번호처럼 항목이 정해져 있고요. 다음 행동도 있습니다. 승인이 나면 설비 명령과 작업지시로 이어지죠.

업무 산출물은 왜 항목이 정해져 있어야 할까요? 사람만 읽는 게 아니라, 시스템이 받아서 다음 일을 하거든요. 항목이 빠지거나 모양이 다르면, 다음 단계가 그 결과를 쓸 수가 없습니다.

[SLIDE 26] 질문에 답하는 에이전트: 답과 근거

두 쪽을 실제 예로 하나씩 볼게요. 먼저 왼쪽, 단순 답변입니다. 에이전트에게 완제품 재고가 가장 적은 설비와 그 수량을 물었어요.

화면의 답을 보면, 2호기 설비이고 완제품 재고가 100개라는 내용입니다. 그런데 이 답은 에이전트가 기억으로 말한 게 아니에요. 업무 DB를 조회해서 얻은 값입니다.

에이전트는 도구를 두 번 불렀어요. 처음에는 어떤 표가 있는지를 확인했고, 그다음에 조회문을 실행했습니다. 이 순서에도 까닭이 있어요. 에이전트에게는 표와 열의 이름을 짐작해서 조회문을 쓰지 말고, 먼저 실제로 있는 표를 확인하라는 규칙이 있거든요. 그래서 확인 한 번, 조회 한 번이 된 겁니다.

답 아래 붙은 게 그 조회문이에요. 완제품 재고 표에서 수량이 적은 순으로 정렬해 읽었죠. 돌아온 몇 행을 적은 순으로 놓았으니, 맨 윗줄이 답이 된 거고요.

답할 수 없을 때도 형식이 정해져 있습니다. 지어내지 않고, 답할 수 없다는 사실과 그 이유를 내요. 답이 한 번으로 끝나더라도 에이전트는 도구로 읽은 값으로 답하고, 어디서 읽었는지를 근거로 남기는 겁니다.

[SLIDE 27] 쿨러 사건의 업무 산출물: 추천안에 담긴 것

이번엔 오른쪽, 업무 산출물입니다. 쿨러 사건의 추천안을 펼쳐 볼게요. 표의 네 줄이 추천안에 들어 있는 네 가지예요.

첫 줄은 원인입니다. 쿨러 핀 오염이고, 고장 유형은 쿨러 냉각 성능 상실이에요. 둘째 줄이 추천이고요. 팬 최대 운전이 1순위였습니다.

셋째 줄은 뒤진 대안 셋이에요. 부하를 낮추는 두 안이 같은 점수로 공동 2순위였습니다. 하나는 부하 70 %에 야간 핀 세척, 다른 하나는 팬 최대 운전에 부하 80 %예요. 원인을 없애는 쿨러 핀 세척은 4순위였고요. 이 점수와 순위는 조치 후보를 견줄 때 나온 결과 그대로입니다.

여기서 고개를 갸웃할 수 있어요. 원인을 없애는 조치가 왜 꼴찌일까요? 핀 세척은 작업지시를 내서 사람이 해야 하는 일이라, 지금 오르고 있는 유온을 당장 내리지는 못하거든요. 그래서 당장 유온을 잡는 조치들보다 뒤에 놓인 겁니다.

넷째 줄은 설명과 판단 번호예요. 추천이 왜 앞서고 대안이 왜 뒤지는지를 설명하고, 이 판단을 가리키는 번호를 붙였습니다.

여기서 중요한 건, 대안이 왜 뒤졌는지까지 함께 낸다는 점이에요. 담당자는 추천 하나만 보고 고르는 게 아닙니다. 무엇과 견주어서 왜 이 안이 나왔는지를 보고 결정할 수 있죠.

[SLIDE 28] 추천안이 넘어가는 곳: 승인 뒤의 명령과 작업지시

이 추천안이 제출된 뒤에 어디로 넘어가는지, 시각과 함께 놓았습니다. 다섯 칸이 에이전트, 사람, 시스템, 세 영역으로 묶여 있어요.

에이전트의 일은 맨 왼쪽 칸에서 끝납니다. 추천안을 내고 단계를 끝냈죠. 다음 칸이 사람의 영역이에요. 잠시 뒤 운전원이 추천안을 승인했습니다.

그 뒤는 전부 시스템이 합니다. 승인 1초 뒤에 팬 속도를 최대로 올리는 설비 명령이 나갔어요. 그 다음 재관측에서 유온이 내려가고 경보가 해제된 걸 확인했고요. 마지막으로 1분쯤 뒤에 핀 세척 작업지시가 등록됐습니다.

명령을 보낸 뒤에도 시스템의 일은 이어집니다. 재관측으로 조치가 정말 듣는지를 확인하죠. 그리고 유온이 내려갔어도 핀은 여전히 오염된 채예요. 경보가 꺼진 것과 원인이 없어진 건 다르니까, 마지막에 핀 세척 작업지시가 붙는 겁니다. 앞에서 4순위였던 세척이 버려진 게 아니라, 승인 뒤의 작업으로 이어진 거죠.

이 쿨러 업무에서 에이전트가 맡은 범위는 조사와 평가, 추천안 제출까지입니다. 추천안은 저장하지만 설비 제어 명령을 직접 내지는 않아요. 운전원이 승인한 뒤 시스템 단계가 명령과 작업지시를 이어 갑니다. 모든 에이전트의 공통 제약이 아니라, 이 업무에서 정해 둔 역할과 권한의 경계입니다.

앞에서 본 두 기능은 모두 에이전트를 활용합니다. 하나는 확인한 답을 사람에게 돌려주고, 다른 하나는 정해진 형식의 제안을 승인 절차로 넘겨요. 어떤 결과를 어디에 넘겨야 하는지 정해야, 에이전트의 완료 조건도 분명해집니다.

이어서 에이전트가 이 일을 하면서 정보와 도구를 어떻게 쓰는지, 그 원리를 봅니다.

[SLIDE 29] 에이전트의 정보원과 도구

이어서 에이전트가 정보와 도구를 쓰는 원리를 봅니다. 앞에서 질문하는 AI와 일하는 AI가 어떻게 다른지 봤죠. 이번에는 에이전트가 그 일을 하면서 무엇을 어디서 읽는지, 언제 멈추는지, 결과와 과정을 어떻게 남기는지로 들어가요. 네 번째 단원은 에이전트의 정보원과 도구입니다. 쿨러 사건의 추천안에는 원인 후보, 측정값, 납기, 작업 절차가 들어 있었어요. 이 정보들은 각각 어디에 있었을까요? 그리고 에이전트는 어떤 길로 그걸 읽었을까요? 정보의 출처와 읽는 길을 차례로 따라가 보겠습니다.

이 원리를 확인한 뒤에는 이어질 두 에이전트 실습에서 무엇을 만들고 어떤 결과를 확인할지도 함께 살펴보겠습니다.

[SLIDE 30] 추천안 한 건에 들어간 정보의 출처

팬 최대 운전, 이 추천안 한 건을 만드는 데 쓰인 정보를 출처별로 놓았습니다. 가운데가 에이전트고, 네 귀퉁이가 서로 다른 역할의 정보예요. 저장소가 반드시 네 개라는 뜻은 아닙니다. HYD에서는 온톨로지와 SOP를 같은 지식 그래프에서 읽어요.

왼쪽 위, 원과 선이 이어진 그림이 온톨로지입니다. 원인 후보 셋이 여기서 왔어요. 오른쪽 위는 시계열 DB고요. 냉각 효율의 측정값이 여기서 왔죠. 오른쪽 아래는 업무 DB예요. 납기까지 남은 시간이 여기 있었습니다. 왼쪽 아래는 SOP예요. 팬 최대 운전의 단계와 그 출처가 여기서 왔습니다.

정보원에서 에이전트로 가는 선을 보면, 선마다 작은 스패너가 있죠. 도구를 뜻합니다. 네 곳 모두 에이전트가 도구를 불러서 읽었다는 표시예요.

이 넷 가운데 하나만 빠져도 추천안은 성립하지 않아요. 원인 후보가 없으면 무엇을 확인할지 모르고, 측정값이 없으면 후보를 가를 수 없죠. 납기를 모르면 생산을 줄이는 안이 얼마나 손해인지 따질 수 없고, SOP가 없으면 조치를 어떻게 수행할지 적을 수가 없습니다.

판단에 쓰는 정보가 여러 곳에 흩어져 있다는 문제, 앞에서 본 적이 있죠. 에이전트는 그 흩어진 정보를 도구로 모아서 추천안 하나로 엮은 겁니다.

[SLIDE 31] 네 정보원이 답하는 물음

그런데 네 정보원은 답하는 물음이 서로 다릅니다. 물음에 맞지 않는 곳을 읽으면 어떻게 될까요? 답이 없거나, 낡은 답을 얻게 돼요.

카드를 하나씩 볼게요. 온톨로지는 무엇과 무엇이 어떤 관계인지에 답합니다. 설비, 증상, 원인, 조치의 관계를 담고 있죠. 시계열 DB는 그 시각의 값이 얼마였는지에 답해요. 센서의 측정값이 시간 순서로 쌓여 있습니다.

업무 DB는 정비, 재고, 납기가 어떤 상태인지에 답합니다. 업무 시스템의 기록이 들어 있어요. 마지막 SOP는 조치를 어떻게 수행하는지에 답하고요. 단계와 승인 역할, 근거가 되는 문서의 절을 정해 둡니다.

물음을 엉뚱한 곳에 던지면 이렇게 됩니다. HYD에서 유온의 현재 측정값을 온톨로지만으로 찾으려 하면, 필요한 값이 없어요. 반대로 원인 후보가 뭔지를 시계열 DB에 물어도 답이 없고요.

처음에 말한 낡은 답은 언제 생길까요? 계속 바뀌는 값을 잘 바뀌지 않는 쪽에 적어 두고 읽을 때예요. 예를 들어 재고 수량을 문서에 적어 두었다면, 문서가 개정될 때까지 그 숫자는 그대로겠죠. 그사이 실제 재고는 계속 바뀌는데요. 그래서 지식 쪽에는 값이 아니라 값을 읽을 자리를 적어 둔다고 했던 겁니다.

카드마다 붙은 꼬리표도 보세요. 그 정보원의 내용이 바뀌는 때입니다. 온톨로지와 SOP는 문서 개정이나 지식 검토를 거쳐 갱신합니다. 시계열 DB는 계속 바뀌고, 업무 DB는 업무가 일어날 때마다 바뀌죠. 이 차이는 뒤에서 지식과 값을 가를 때 다시 씁니다.

[SLIDE 32] 지식 그래프에서 읽는 것: 원인 후보와 조치 방법

네 곳에서 실제로 뭘 읽었는지 하나씩 볼게요. 그 전에 기억해 둘 게 있어요. 네 정보원 가운데 온톨로지와 SOP는 같은 지식 그래프에 들어 있습니다. 지금까지 함께 만들어 온 바로 그 그래프예요.

화면 아래 가운데 진한 원이 고장 유형, 냉각 성능 상실입니다. 위쪽 원인 셋에서 화살표가 이 고장 유형으로 모이죠. 쿨러 핀 오염, 팬 구동 불량, 주변 온도 상승, 이 셋이 이 고장을 일으킬 수 있다는 관계예요. 그리고 오른쪽 아래로는 고장 유형에서 조치 방법으로 가는 관계가 있습니다. 이 고장을 완화하는 방법이 팬 최대 운전이라는 뜻이죠.

선 옆에 작게 적힌 게 관계 이름이에요. 위쪽 세 선은 일으킨다는 관계, 오른쪽 아래 선은 완화한다는 관계입니다. 진단 도구는 고장 유형에서 원인 관계를 거슬러 올라가서 후보 셋을 얻었어요. 완화 관계를 따라가서는 조치 방법을 얻었고요.

이 그래프가 왜 여기서 쓸모 있을까요? 경보는 어떤 고장이 의심된다는 것까지만 알려 줘요. 그런데 관계를 거슬러 가면 확인해야 할 원인 목록이 바로 나옵니다. 에이전트가 원인 후보를 짐작으로 늘어놓을 필요가 없는 거죠. 게다가 이 관계들은 매뉴얼에서 뽑아 사람이 검토하고 등록한 지식이에요. 그래서 에이전트가 내놓는 원인 후보는 검토를 거친 지식 안에서만 나옵니다.

원인 이름표의 둘째 줄도 보세요. 원인마다 사전 확률이 적혀 있죠. 문서에서 지식을 뽑을 때 봤던 값으로, 매뉴얼이 이 원인을 얼마나 흔하다고 보는지를 옮긴 거예요. 셋 가운데 쿨러 핀 오염이 가장 흔합니다.

그래도 한 가지 짚어 둘게요. 흔하다는 게 이번 사건의 원인이라는 뜻은 아니에요. 그래프가 알려주는 건 후보와 그 흔한 정도까지입니다. 셋 가운데 어느 원인이 이번 사건에 맞는지는 그래프만 봐서는 알 수 없어요. 그걸 가르려면 값을 읽어야 합니다.

[SLIDE 33] 시계열 DB에서 읽는 것: 판단하는 순간의 측정값

그 값을 읽는 곳이 시계열 DB예요. 화면 왼쪽은 원인 진단이 확인한 증거 한 건입니다. 여기서 읽은 값 하나를 자세히 들여다볼게요.

첫 줄은 읽은 방법입니다. 최근 30초 동안 냉각 효율의 평균을 읽으라는 거죠. 이 방법은 에이전트가 지어낸 게 아니에요. 지식 그래프의 증거에 적혀 있던 그대로입니다. 앞에서 원인 진단의 물음에는 세 값뿐이었다고 했죠. 무엇을 어떻게 읽을지는 바로 여기, 증거 쪽에 적혀 있었던 겁니다.

구간 평균은 한 번 튄 값의 영향을 줄여서 볼 때 쓸 수 있어요. 반면 짧은 이상을 놓칠 수도 있죠. 그래서 언제나 평균이 더 좋은 것은 아닙니다. 이 사례에서는 등록된 증거 기준이 최근 30초 평균을 요구하므로 그 기간과 집계 방식을 맞춰 읽습니다.

그 아래 측정값은 약 46퍼센트, 기준은 70퍼센트 미만입니다. 앞의 값은 시계열 DB에서 읽었고, 뒤의 기준은 지식 그래프에서 가져왔어요. 이 측정값은 기준보다 낮으므로 해당 증거 조건을 만족합니다. 원인 전체의 판정은 다른 증거와 함께 봐야 하고요.

맨 아래에는 관측 범위에 관한 정보도 붙어 있습니다. 표본이 30개였다는 기록이 보여요. 다만 개수만으로 매초 빠짐없이 들어왔다고 단정하면 안 됩니다. 관측 시각과 비어 있는 구간도 확인해야 하죠. 평균값을 얻었더라도 관측 공백이 크면 그 구간을 대표한다고 보기 어렵기 때문입니다. HYD의 진단도 이런 데이터 품질 조건을 함께 확인합니다.

[SLIDE 34] 업무 DB에서 읽는 것: 설비 밖의 사정

다음은 업무 DB예요. 납기나 남은 수량은 설비에 달린 센서로는 알 수가 없죠. 이런 값은 업무 DB에 있습니다.

화면 왼쪽은 앞에서 본 다른 실행의 기록에서 꺼낸 생산오더 조회예요. 위쪽이 물음인데, 에이전트는 설비 이름 하나만 넣었습니다. 그 아래가 돌려받은 결과고요. 오른쪽 첫 설명을 보면, 값이 온 업무 시스템을 표시한 칸이에요. 생산 일정을 관리하는 MES입니다.

둘째 설명은 판단에 쓰는 값들을 모아 놓은 칸이에요. 이 설비에 걸린 생산오더는 납기까지 약 6시간이 남았고, 아직 만들어야 할 수량도 꽤 남아 있었습니다. 같은 방식으로 품질 쪽에서는 출하를 기다리는 로트를 읽었고요. 오른쪽 셋째 설명이 그거예요. 이 로트는 품질 쪽 재료입니다. 유온이 높은 채로 이어지면 출하를 앞둔 제품에 품질 문제가 생길 위험이 커지는데, 그 위험을 따지는 데 쓰여요.

쿨러 사건의 실행에서는 이 조회를 따로 부르지 않았죠. 대안 평가가 같은 생산오더의 납기를 평가 안에서 읽었어요. 그 값이 조치를 견줄 때 쓰였습니다. 부하를 낮추는 조치는 유온을 더 내리지만 생산이 줄어요. 납기까지 6시간도 안 남은 상황이라, 그 손실이 성과 비교에서 점수로 드러난 거죠. 만약 유온을 많이 내리는 정도만 비교했다면 그 효과가 큰 안을 앞세웠겠죠. 실제로는 그 안이 공동 2순위였죠. 설비 밖의 사정이 설비 조치의 순위를 바꾼 겁니다.

[SLIDE 35] HYD의 데이터베이스 접근: 읽기 전용 도구 경유

그럼 에이전트에게 데이터베이스를 자유롭게 열어 주어야 할까요? HYD는 그 사이에 도구를 두었습니다. 화면의 에이전트는 도구에 물음을 보내고, 도구가 제한된 권한으로 읽어 온 결과를 받습니다. 데이터베이스를 직접 연결하는 구성도 가능하지만, 여기서는 접근 경로를 도구로 관리하는 방식을 택한 거예요.

가운데 자물쇠가 그 제한을 나타냅니다. 조회 도구는 읽기 전용 계정을 사용하므로, 잘못된 조회 요청이 데이터를 수정하거나 지우는 작업으로 실행되지 않게 합니다. 조회 결과의 양과 실행 시간에도 한도를 둡니다. 화면의 200행과 5초는 이 시스템의 SQL 조회 설정이고, 모든 도구에 공통인 숫자는 아니에요.

결과의 행 수를 제한하는 것과 데이터베이스가 계산할 양을 제한하는 것은 다릅니다. 한 줄짜리 합계를 얻으려고도 많은 데이터를 읽을 수 있거든요. 시간 제한 같은 장치를 함께 두어 자원 사용을 줄이지만, 조회가 항상 가볍고 안전하다고 보장할 수는 없습니다. 질문에 맞는 기간과 범위를 정하는 일도 필요해요.

또한 읽기 도구의 제한을 에이전트의 모든 행동과 혼동하면 안 됩니다. 추천안 제출 도구는 기록을 남깁니다. 설비 제어 명령은 또 별도의 권한과 승인 절차를 거치고요. 무엇을 읽을 수 있고 무엇을 쓸 수 있는지는 도구와 업무마다 정해야 합니다.

정보를 얻는 길이 정해졌습니다. 다음에는 제대로 읽어 온 정보라도 잘못 해석할 수 있는 지점, 기준과 측정값의 구분을 살펴보겠습니다.

[SLIDE 36] 지식과 현재값의 구분

앞에서 정보원마다 붙은 꼬리표를 보면서, 잘 바뀌지 않는 지식과 계속 바뀌는 값이 나뉜다고 했죠. 그 둘을 가르는 게 다섯 번째 단원, 지식과 현재값의 구분입니다. 제대로 읽은 정보라도, 에이전트가 이 둘을 섞어 쓰면 어떤 일이 생길까요? 섞이는 꼴부터 보고, 그걸 막는 장치 두 가지를 차례로 보겠습니다.

[SLIDE 37] 지식과 현재값을 섞는 두 가지 꼴

지식과 현재값이 섞이는 꼴은 두 가지예요. 화면의 두 칸에 틀린 문장의 예를 하나씩 놓았습니다. 따옴표 안이 그 틀린 문장이에요.

왼쪽 문장은 냉각 효율이 70 %라고 말합니다. 그런데 70은 매뉴얼에 적힌 판별 기준이죠. 잰 값이 아니에요. 실제로 잰 값은 약 46퍼센트였습니다. 기준을 측정값처럼 말한 경우예요.

이 혼동이 판단을 어떻게 바꾸는지 따라가 볼까요? 기준은 70 미만이었죠. 이 70을 잰 값으로 알고 기준에 대면, 70은 70 미만이 아니니까 냉각 효율이 낮다는 증거는 맞지 않는 걸로 판정됩니다. 쿨러 핀 오염을 뒷받침하던 증거 하나가 엉뚱하게 사라지는 거예요.

오른쪽 문장은 유온이 약 56도이라서 한계까지 여유가 있다고 말합니다. 그런데 그 값은 경보가 난 순간에 읽은 값이에요. 운전원이 승인하던 순간의 유온은 62도 가까이로 올라 있었습니다. 한 번 읽은 값을 한참 뒤에도 지금의 값처럼 쓴 경우죠. 그사이 한계까지의 여유는 꽤 줄어 있었어요. 여유가 있다고 믿으면 덜 급한 조치를 골라도 된다는 판단으로 흐르기 쉽습니다.

두 문장 다 숫자가 들어 있어서 그럴듯하게 읽힙니다. 문제는 읽는 사람이 알아차리기 어렵다는 거예요. 그래서 이 혼동은 사람이 잡아내길 기다릴 게 아니라, 에이전트가 처음부터 하지 않도록 막아야 합니다.

[SLIDE 38] 혼동을 막는 작업 규칙: 추측 대신 도구로 확인한다

그걸 막는 첫째 장치가 작업 규칙입니다. 에이전트에게는 추측하지 말고 도구로 확인하라는 규칙이 주어져요. 무엇을 어디서 얻을지가 화면처럼 세 갈래로 정해져 있습니다.

첫째 카드는 지식이에요. 원인 후보, 증거의 기준, 조치 방법은 지식 그래프를 조회해서 얻습니다. LLM만으로는 이 공장의 기준을 몰라서 일반론으로 대신 답한다고 했죠. 그 빈자리를 메우는 게 이 카드예요. 모델이 원래 알고 있는 일반 지식으로 기준을 말하지 않고, 이 공장의 기준을 그래프에서 꺼내 씁니다. 둘째 카드는 지금의 값이고요. 센서 값, 납기, 재고는 시계열 DB와 업무 DB에서 그때그때 읽어요. 기억해 둔 값을 쓰지 않는 거죠. 이 카드의 마지막 줄도 보세요. 값마다 어디서 어떻게 읽었는지가 함께 붙습니다. 쿨러 사건에서도 유온에는 시계열 DB의 최신값이라는 출처가, 긴급 오더의 남은 시간에는 MES라는 출처가 붙어 있었어요. 그래서 나중에 이 값이 어디서 왔는지를 따져 볼 수 있죠.

셋째 카드는 규칙 판정입니다. 어떤 조치가 제외되는지, 감점이나 경고가 붙는지, 이건 모델이 해석하지 않아요. 판단 도구가 계산한 결과를 씁니다.

규칙 판정은 왜 도구에 맡길까요? 모델에게 규칙을 해석하게 하면, 같은 상황에서도 답이 달라질 수 있거든요. 판단 도구는 같은 판본의 규칙에 같은 입력을 넣으면 같은 방식으로 계산합니다. 그래야 판정을 믿을 수 있고, 다시 확인할 수도 있죠.

물론 규칙을 글로 주었다고 모델이 반드시 지키는 것은 아닙니다. 도구의 권한 제한, 데이터 상태 검사, 결과 검토가 함께 필요해요. 지금 본 것은 어떤 출처를 써야 한다는 지시이고, 다음은 그 출처에서 읽은 값의 상태를 검사하는 장치입니다.

[SLIDE 39] 신선도: 관측된 값이 얼마나 오래됐는가

방금 조회에 성공했다고 해서 지금의 상태를 읽었다고 볼 수 있을까요? 센서가 멈췄다면 데이터베이스에는 오래전 값이 마지막 값으로 남아 있을 수 있습니다. 그래서 신선도는 조회 버튼을 누른 때가 아니라, 값이 관측되어 기록된 시각이 판단 시점에서 얼마나 오래됐는지를 봅니다.

화면의 굵은 세로선은 판단 시점이고, 그 왼쪽 띠가 허용 범위예요. HYD의 이 검사에서는 최신 유온의 관측 시각이 60초 안쪽인지 봅니다. 60초는 이 시스템의 설정이지 모든 설비의 공통 기준은 아닙니다. 공정이 얼마나 빨리 변하고 어떤 판단에 쓸 값인지에 맞춰 정해야 해요.

쿨러 사건의 검사 시점에는 최신 유온이 1초도 안 된 값이었습니다. 반대로 60초보다 오래된 값만 남아 있었다면 이 검사를 통과하지 못해요. HYD는 수집 상태와 시각의 이상도 함께 확인합니다. 최신 행을 찾았다는 사실만으로 충분하지 않은 거죠.

검사에 통과한 값도 시간이 지나면 오래됩니다. 조사와 평가가 길어졌다면 중요한 실행 전에 다시 확인할 필요가 있어요. 다만 지금 화면은 한 검사 시점의 결과입니다. 모든 제출과 승인 순간에 자동으로 재검사됐다고 이 기록만으로 말할 수는 없습니다. 실제 업무를 설계할 때는 무엇을 어느 단계에서 다시 확인할지도 정해야 합니다.

[SLIDE 40] 멈춤 조건: 근거 부족 · 호출 실패 · 권한 제한

방금 값이 묵었으면 판단을 보류한다고 했죠. 일을 끝까지 하는 것보다 멈추는 게 맞을 때가 있다는 뜻이에요. 여섯 번째 단원은 바로 그 멈춤 조건입니다. 에이전트는 언제 멈춰야 하고, 멈출 때는 무엇을 내야 할까요?

[SLIDE 41] 멈추지 않는 에이전트가 만드는 결과

먼저 에이전트가 멈추지 않으면 어떻게 되는지부터 볼게요. 냉각 효율을 읽지 못한 상황 하나에서 나올 수 있는 두 결과를 나란히 놓았습니다.

왼쪽은 빈칸을 채워서 제출하는 경우예요. 냉각 효율을 못 읽었는데, 사전 확률이 가장 큰 쿨러 핀 오염을 원인으로 적어 냈습니다. 앞에서 본, 가장 흔하다는 그 값만 믿고 적은 거죠. 그런데 이 추천안, 모양은 값을 읽었을 때와 똑같아요. 담당자는 확인된 원인이라고 믿고 승인하겠죠. 원인이 틀렸더라도 겉으로는 드러나지 않습니다. 문제는 원인이 다르면 맞는 조치도 달라진다는 거예요.

쿨러 핀이 오염된 거라면 핀을 세척해야 하지만, 팬이 제대로 돌지 않는 거라면 팬 쪽을 점검해야 하죠. 틀린 원인 위에 세운 추천안은 엉뚱한 정비로 이어질 수 있습니다.

오른쪽은 멈추고 이유를 내는 경우예요. 원인을 적지 않고 보류합니다. 그리고 무엇을 확인하지 못했는지를 적어 내죠. 그러면 담당자는 값이 들어온 뒤에 다시 평가하게 할 수 있어요. 모르는 게 모른다는 모습 그대로 드러나는 겁니다.

멈춘다는 건 일을 못 한 게 아니에요. 승인하는 사람이 없는 근거를 믿고 결정하지 않게 하는 일입니다. 그래서 어떤 때 멈출지를 미리 정해 둬요.

[SLIDE 42] 멈춤 조건 세 가지

에이전트가 멈춰야 하는 대표적인 상황은 셋입니다. 화면 왼쪽 에이전트에서 세 갈래로 갈라지죠. 위에서부터 볼게요.

위 갈래는 근거 부족이에요. 값을 얻지 못했거나, 얻은 값이 어느 원인의 기준에도 맞지 않는 경우입니다. 가운데 갈래는 도구 호출 실패고요. 다시 시도해도 도구가 오류를 돌려주는 경우죠. 아래 갈래는 권한 제한입니다. 일을 끝내려면 허용되지 않은 도구가 필요한 경우예요.

갈래 끝마다 그 상황을 뜻하는 그림이 있어요. 근거 부족에는 값이 빈 저장소, 호출 실패에는 가위표가 붙은 도구, 권한 제한에는 자물쇠가 그려져 있죠.

세 갈래 모두 멈춤 표지를 지납니다. 에이전트는 지어내지 않고 멈추는 거죠. 그리고 오른쪽의 사람에게 모여요. 사람은 다시 평가하게 하거나, 에이전트의 질문에 답하거나, 작업을 닫습니다. 오른쪽 사람 이름표의 둘째 줄이 그 세 가지예요.

세 상황의 공통점은, 에이전트가 근거를 갖고 끝까지 갈 수 없다는 거예요. 그럴 때 할 일은 그럴듯한 결과를 만드는 게 아닙니다. 이유와 함께 사람에게 넘기는 거죠.

[SLIDE 43] 근거의 세 가지 상태: SUPPORTED · UNKNOWN · UNSUPPORTED

세 조건 가운데 근거 부족부터 자세히 볼게요. 근거 부족에는 사실 서로 다른 두 경우가 들어 있어요. 그래서 근거의 상태를 화면처럼 셋으로 나누어 적습니다.

왼쪽 카드는 근거가 지지하는 상태예요. 필요한 값을 읽었고 등록된 증거 기준에 맞았습니다. 냉각 효율이 약 46퍼센트로 70퍼센트 미만이었다는 것은 그중 한 조건이에요. 쿨러 핀 오염이라는 후보의 판정은 팬 진동을 비롯한 필요한 증거와 함께 봅니다. 지지된다는 말은 이 모델과 기준 안에서의 판정이지, 현장의 모든 가능한 원인을 배제했다는 뜻은 아닙니다.

가운데 카드는 확인하지 못한 상태입니다. 조회에 실패했거나 값이 비어서, 맞는지 틀린지를 말할 수가 없어요. 결정표를 다룰 때도 값을 얻지 못한 조건을 통과로 치지 않았죠. 여기서도 같은 생각이에요. 모르는 것을 맞다고도, 틀리다고도 하지 않고 모른다고 적는 겁니다.

오른쪽 카드는 맞는 근거가 없는 상태고요. 값은 읽었는데, 조회한 근거가 모두 기준에 맞지 않았습니다. 등록된 원인 가운데 맞는 게 하나도 없는 거죠.

가운데와 오른쪽을 굳이 나누는 까닭은, 사람이 이어서 할 일이 다르기 때문이에요. 확인하지 못한 경우는 필요한 값이 들어오면 다시 평가할 수 있습니다. 맞는 근거가 없는 경우는 다르죠. 지식 그래프에 등록되지 않은 원인일 수 있어요. 그러면 사람이 직접 살펴야 합니다.

[SLIDE 44] 보류의 형식: 상태 · 이유 · 증거

상태를 나눴으면, 보류할 때 내는 형식도 정해져 있어야겠죠. 화면 왼쪽이 보류할 때 에이전트가 내는 객체예요. 작업 지시에 적혀 있는 형식 그대로인데, 항목이 세 개뿐입니다.

첫째 칸은 상태예요. 방금 본 확인하지 못함과 맞는 근거 없음 가운데 하나를 적습니다. 둘째 칸은 이유고요. 무엇을 확인하지 못했는지를 사람이 읽을 문장으로 적어요. 예를 들면 유온의 최신 값이 60초보다 오래되었다, 이런 식이죠. 보류된 작업은 결국 사람이 받으니까, 사람이 읽고 바로 무엇을 해야 할지 알 수 있는 문장이어야 합니다.

셋째 칸은 그때까지 조회해서 얻은 실제 근거입니다. 멈췄다고 해서 그때까지 얻은 걸 버리지 않아요. 그래야 사람이 다시 평가를 시킬 때, 무엇은 이미 확인됐고 무엇이 비었는지를 한눈에 볼 수 있죠.

화면 아래도 보세요. 원인, 고장 유형, 판단 같은 결과 항목은 보류와 함께 낼 수 없습니다. 섞어서 내면 거절돼요.

섞어 내기를 막아 두지 않으면 어떻게 될까요? 일부만 채운 결과가, 절반만 확인된 추천안으로 승인 화면에 올라가게 됩니다. 그래서 보류는 결과를 하나도 채우지 않고, 이유와 근거만 넘기는 거예요.

[SLIDE 45] 조회 실패 · 일치하는 기록 없음 · 수량 0

도구가 오류를 돌려준 경우와 조회는 됐지만 찾은 행이 없는 경우도 구분해야 합니다. 둘을 같은 빈칸으로 처리하면, 확인하지 못한 일을 없는 일로 바꿔 버릴 수 있어요.

예비품 재고를 조회한다고 해 볼게요. 왼쪽은 조회에 성공했지만 조건에 맞는 기록이 0행인 경우입니다. 이때 알 수 있는 것은 그 조회 범위 안에서 일치하는 기록을 찾지 못했다는 사실이에요. 실제 창고의 재고가 없다고 바로 단정할 수는 없습니다. 창고나 품목 조건을 잘못 넣었거나, 해당 재고가 아직 등록되지 않았을 수도 있죠.

재고 행을 찾았고 그 행의 수량이 0인 경우는 또 다릅니다. 조회한 대상의 기록상 수량이 0이라는 답을 받은 거예요. 어느 창고와 어느 시점의 기록인지까지 맞아야 질문의 답으로 쓸 수 있습니다.

오른쪽은 조회 자체가 실패한 경우입니다. 수량도, 조건에 맞는 기록의 유무도 확인하지 못했어요. 이 실패를 0으로 바꾸면, 읽지 못한 재고를 없다고 믿고 새로 발주할 수도 있습니다. 그래서 실패, 일치하는 기록 없음, 기록된 수량 0을 각각 그대로 남겨야 해요. 필수 값을 끝내 확인하지 못했다면 판단을 보류합니다.

[SLIDE 46] 재시도: 반복의 영향과 한도

방금 다시 시도해도 끝내 실패하면 실패했다고 그대로 적는다고 했죠. 그럼 끝내라는 건 언제일까요? 에이전트가 끝없이 다시 시도할 수는 없어요. 다시 시도하는 데는 한도가 있습니다. 화면의 카드 셋이 그 한도고, 카드 아래 꼬리표에 한도나 그걸 정하는 쪽이 적혀 있어요.

첫째 카드는 도구를 다시 부르는 일이에요. 앞에서는 결과가 너무 커서 읽는 방법을 바꾸겠다는 기록을 봤죠. 다시 시도할 때는 먼저 읽기인지 쓰기인지 구별해야 합니다. 조회를 다시 하는 것과 추천안이나 발주를 다시 제출하는 것은 영향이 다르거든요. 제출 응답이 늦었다고 해서 저장까지 실패한 것은 아닐 수 있어요. 같은 요청을 무작정 보내면 두 건이 생길 수 있습니다. 기존 처리 여부를 확인하거나 같은 요청의 중복 반영을 막는 장치가 필요합니다.

둘째 카드는 조회문 수정이에요. 질문에 답하는 에이전트가 조회문을 직접 써서 실행했던 것, 앞에서 봤죠. 그렇게 직접 쓴 조회문이 오류를 내면 고쳐서 다시 실행할 수 있는데, 두 번까지 시도하라고 작업 지침에 적혀 있습니다. 이 숫자는 모델에게 주는 지시이고, 그 자체가 서버의 강제 차단을 뜻하지는 않아요. 고칠 때 지키는 게 하나 있어요. 오류가 난 부분만 고치고, 무엇을 세고 무엇을 거를지는 바꾸지 않습니다. 오류를 없애려고 묻는 내용 자체를 바꿔 버리면, 오류는 사라져도 엉뚱한 물음에 대한 답을 얻게 되니까요. 두 번을 고쳐도 안 되면, 방금 본 대로 실패했다고 그대로 적습니다.

셋째 카드는 실행 시간이에요. 확인한 실행 설정에는 한 번의 실행 전체에 30분의 제한이 걸려 있습니다. 꼬리표에는 초로 적혀 있죠. 이것도 이 시스템에 정해 둔 값이에요. 이 시간을 넘기면 작업은 멈춘 채로 남고, 사람이 닫아야 해요. 시간이 다 된 경우도 멈춤 조건과 같은 자리에 있는 셈입니다. 근거를 갖고 끝까지 가지 못했으니, 결과를 지어내지 않고 멈추는 거죠.

한도는 왜 둘까요? 에이전트가 같은 실패를 끝없이 되풀이한다고 생각해 보세요. 결과는 안 나오고 시간만 흐르겠죠. 사람은 그동안 아무것도 알 수가 없고요. 그래서 한도를 넘긴 작업은 멈춰서 사람에게 넘어갑니다.

[SLIDE 47] 권한 제한: 허용 목록 밖의 도구

마지막 멈춤 조건은 권한 제한이에요. 출발점은 화면 첫 칸의 허용 목록입니다. HYD는 에이전트의 도구 호출을 이 허용 목록으로 제한합니다.

둘째 칸을 보세요. 에이전트가 목록 밖의 도구를 부르려고 하면, 그 호출은 실행되지 않고 권한 요청이 됩니다. 셋째 칸, 그 요청은 다시 질문이 되어 사람에게 올라가요. 에이전트가 일을 하려면 꼭 필요하다고 판단했더라도 마찬가지입니다.

마지막 칸은 에이전트에게 주는 지시의 한 문장이에요. 답변은 실행 권한을 바꾸지 않는다. 사람이 그 질문에 답하더라도, 그 답으로 에이전트의 권한이 넓어지지는 않는다는 뜻입니다.

왜 답변으로 권한이 바뀌면 안 될까요? 무엇을 부를 수 있는지는 에이전트를 정의할 때 미리 정해 둔 거예요. 맡긴 업무에 필요한 기능과 권한을 미리 검토해서 등록하는 것이죠. 실행 중에 오간 질

문과 답으로 그게 바뀐다면, 미리 정해 둔 목록은 의미가 없어지겠죠. 권한을 넓히는 일은 에이전트의 정의를 고치는 일이지, 일하는 도중의 대화로 할 일이 아닌 겁니다.

여기서 에이전트가 해서는 안 되는 일이 하나 있어요. 허용된 도구를 이리저리 엮어서, 막힌 일을 돌아가는 겁니다. 지시에도 사람에게 묻는 질문을 도구 거절이나 승인 우회에 쓰지 말라고 적혀 있어요.

세 조건을 다 봤으니 이 단원의 물음에 답해 볼까요? 에이전트는 언제 멈추고 무엇을 내는가. 근거가 모자랄 때도, 호출이 끝내 실패할 때도, 권한 밖의 일이 필요할 때도 할 일은 같습니다. 결과를 채우지 않고 멈춰서, 이유와 근거를 붙여 사람에게 넘기는 거죠.

[SLIDE 48] 구조화 출력과 실행 기록

멈출 때 내는 보류에는 정해진 형식이 있었죠. 멈추지 않고 끝까지 간 결과에도 형식이 있습니다. 그걸 다루는 게 에이전트 원리를 마무리하는 단원, 구조화 출력과 실행 기록이에요. 사람이 추천안을 믿고 확인하려면, 결과와 과정이 어떤 모양으로 남아야 할까요? 결과를 정해진 형식으로 내는 구조화 출력부터 보고, 일한 과정을 남기는 실행 기록, 그리고 확인된 원인과 가설을 가르는 것까지 보겠습니다.

[SLIDE 49] 구조화 출력: 결과 객체의 다섯 항목

결과 쪽부터 볼게요. 화면 왼쪽은 쿨러 사건에서 에이전트 단계가 끝날 때 남긴 결과의 뼈대예요. 종료 장면에서 작업 공간에 썼다던 그 결과 파일이죠. 다섯 항목을 가진 JSON 객체 하나입니다. 문서에서 지식을 뽑을 때 썼던 구조화 출력이 여기서도 쓰인 거죠.

맨 위 두 항목은 원인과 고장 유형이에요. 이름이 아니라 지식 그래프의 식별자로 적습니다. 셋째 항목은 진단 안내고요. 진단의 결과와 권장 조치를 담고 있어요.

넷째는 판단 항목입니다. 오른쪽 설명에서 강조한 항목이죠. 추천하는 조치, 그 이유의 설명, 후보의 순서가 들어 있어요. 앞에서 추천안의 표로 본 추천, 이유, 뒤진 대안이 바로 이 안에 있는 거죠. 이 가운데 설명은 승인하는 사람이 읽는 부분입니다. 마지막 항목은 판단 제출 도구에서 돌려받은 판단 번호예요. 판단 제출로 낸 추천안과 이 결과가 이 번호로 이어집니다.

여기서 헷갈리기 쉬운 게 하나 있어요. 오른쪽 셋째 설명의 끝줄처럼, 항목 칸에는 식별자를 적지만 설명 문장에는 식별자 대신 팬 최대 운전처럼 이름을 씁니다. 칸은 다음 시스템이 받아서 정확히 찾아가야 하니까 식별자고, 문장은 사람이 읽으니까 이름인 거죠.

어떤 항목이 있어야 하는지는 작업마다 미리 정해 두는데, 이걸 출력 계약이라고 해요. 승인 화면은 판단 항목에서 추천과 설명을 꺼내 보여 주고, 뒤의 단계는 판단 번호로 이 판단을 가리킵니다. 항목이 정해져 있어야, 받는 쪽이 어디서 무엇을 꺼낼지 알 수 있는 거죠.

[SLIDE 50] 출력 형식의 검사와 교정

그럼 형식이 어긋나면 어떻게 될까요? 에이전트가 낸 결과는 시스템이 먼저 검사합니다. 형식이 맞아야 다음 단계로 넘어가요.

화면의 네 칸을 따라가 볼게요. 첫 칸에서 에이전트가 결과 객체를 냅니다. 둘째 칸에서 시스템이 정해진 항목과 맞는지를 보고요. 항목이 빠졌거나 모양이 어긋나면 셋째 칸으로 가요. 출력 형식 검사에서 거절됐다는 알림이, 이유와 함께 에이전트에게 돌아가죠.

에이전트는 같은 실행 안에서 고쳐서 다시 냅니다. 내용은 확인됐고 형식만 어긋난 경우라면, 얻어 둔 내용을 정해진 항목에 맞춰 고치는 겁니다. 형식을 맞추려고 새로운 값을 지어 넣으면 안 되겠죠. 필요한 사실 자체가 빠진 경우는 단순한 모양 수정으로 해결할 수 없습니다. 이 시스템의 형식 교정은 최대 두 번이며, 통과하면 승인할 담당자에게 넘어갑니다.

그런데 이 검사가 보는 건 형식이에요. 항목이 다 있는지, 모양이 맞는지를 봅니다. 형식이 맞는다고 내용까지 옳다는 뜻은 아니죠. 내용이 옳은지는 다른 방법으로 확인해야 합니다.

[SLIDE 51] 실행 기록 한 줄에 남는 것

내용을 확인하는 방법, 그게 과정의 기록이에요. 결과와 함께 과정도 남습니다. 에이전트가 일하는 동안 말과 도구 호출이 시각 순서로 한 줄씩 남는데, 이걸 실행 기록이라고 해요. 화면 위의 상자는 그 실행 기록을 보여 주는 포털의 처리 기록 화면입니다. 앞에서 처리 기록이라고 부르며 본 줄들이 바로 이 실행 기록이었던 거죠.

그 가운데 진하게 표시한 셋째 줄이 도구 호출 한 건입니다. 위 상자의 그 줄을 아래로 끌어내려서 다섯 칸으로 펼쳤어요. 왼쪽부터 언제, 어느 도구에, 무엇을 물어서, 무엇을 받았고, 얼마나 걸렸는지입니다.

앞에서 본 원인 진단 호출도 이 다섯 항목으로 읽을 수 있습니다. 숫자와 식별자를 모두 읽기보다, 지금 확인하려는 값이 어느 입력에서 나와 어떤 결과로 돌아왔는지 연결해 보세요.

여기서 중요한 건 가운데 두 칸이에요. 입력과 출력이 모두 남는다는 점입니다. 그래서 추천안에 적힌 값이 어느 호출에서 왔는지를 찾아갈 수 있어요.

같은 경보를 처리한 두 실행이 서로 다른 길로 일했던 것, 기억하죠. 같은 지침이라도 실행마다 부른 도구와 순서가 달라질 수 있어요. 그래서 결과만 보고는 이번 실행이 어떻게 일했는지 알 수 없고, 실행마다 이 기록이 남아야 하는 겁니다.

[SLIDE 52] 실행 기록의 쓰임: 결과에서 호출로 거슬러 올라가기

이 기록은 언제 쓸까요? 결과가 이상해 보일 때입니다. 화면의 네 단계처럼, 결과에서 출발해 호출로 거슬러 올라가는 거예요.

첫째, 추천안에서 의심스러운 값을 고릅니다. 둘째, 그 값을 돌려준 호출을 기록에서 찾아요. 출력이 그 값과 같은지 확인하는 거죠. 셋째, 그 호출의 입력을 봅니다. 엉뚱한 설비를 넣지 않았는지, 틀린 원인을 넣지 않았는지요.

먼저 도구의 출력과 최종 설명이 같은 뜻인지 봅니다. 모델이 단위를 바꾸거나 다른 호출의 값을 섞었을 수도 있어요. 입력도 질문에 맞고 출력도 그대로 옮겼다면, 그때 도구의 계산과 원천 데이터까지 살펴봅니다. 호출 사이의 진행 설명은 에이전트가 밝힌 의도를 읽는 보조 자료예요. 그 설명만으로 실제 처리나 오류의 원인을 확정하지는 않습니다.

쿨러 사건에 대어 볼까요? 추천안의 원인, 쿨러 핀 오염이 의심스럽다고 해 봅시다. 그 값을 돌려준 호출은 원인 진단이죠. 기록에서 그 호출의 받음을 열어, 원인 1순위가 정말 쿨러 핀 오염인지 봅니다. 맞다면 물음으로 가요. 설비가 HYD-01이 맞는지, 경보 번호가 이 경보가 맞는지. 입력과 반환값, 최종 설명의 연결까지 맞는다면 진단 도구가 읽은 증거와 값 쪽을 더 확인합니다.

요청 해석 장면에서, 에이전트가 진행 설명을 남긴다고 했죠. 그 말이 여기서 쓰이는 거예요. 결과만 있으면 틀렸다는 것밖에 모릅니다. 하지만 과정이 남아 있으면, 어디서 틀렸는지를 찾을 수 있어요.

[SLIDE 53] 근거 제시: 식별자의 인용

과정의 기록과는 별도로, 결과 안에도 근거가 들어 있어야 합니다. 에이전트는 모든 판단에 지식 그래프의 식별자를 인용하게 되어 있어요.

화면의 카드 셋이 쿨러 사건의 추천안에 인용된 식별자입니다. 원인은 쿨러 핀 오염, 고장 유형은 쿨러 냉각 성능 상실, 조치 방법은 팬 최대 운전이죠. 카드마다 붙은 꼬리표는 식별자의 앞머리예요. 원인, 고장 유형, 조치 방법이 각각 다른 앞머리를 갖습니다.

식별자는 다음 시스템이 받아 쓰기 위한 것만이 아니에요. 근거로 쓸 때도 이름과는 다르죠. 쿨러 핀 오염이라는 이름만 있으면, 그게 등록된 지식인지 모델이 지어낸 말인지 구별할 수가 없어요. 식별자가 있으면 그 노드를 찾아갈 수 있습니다. 조치 방법이라면 출처인 매뉴얼의 절까지 확인할 수 있고요. 강조한 셋째 카드에 붙은 출처가 바로 매뉴얼의 즉시 완화 조치 절이에요. 문서에서 지식을 등록할 때 출처를 함께 남겼던 게, 여기서 에이전트의 근거로 쓰이는 겁니다.

식별자가 적혀 있다는 사실 자체가 근거의 정확성을 보장하지는 않습니다. 실제 노드가 존재하는지, 이번 설비와 연결되는지, 인용한 절이 지금의 주장까지 뒷받침하는지 확인해야 해요. 식별자는 그 확인을 시작할 주소 역할을 합니다.

[SLIDE 54] 근거의 대조: 적어 낸 근거가 호출 기록에 있는가

그렇다고 에이전트가 근거를 적었다는 것만으로 충분할까요? 아닙니다. 근거 자체를 꾸며 낼 수도 있으니까요. 그래서 시스템이, 적어 낸 근거를 실제 호출 기록과 대조합니다.

질문에 답하는 경우로 볼게요. 앞에서 완제품 재고를 물었을 때, 답 아래에 조회문이 근거로 붙어 있었죠. 시스템은 바로 그 조회문을 호출 기록과 대어 봅니다. 왼쪽은 호출 기록에 있는 근거예요.

답에 적힌 조회문이 실제로 실행한 조회와 같습니다. 그러면 화면에 근거 몇 개가 확인됐다고 표시되고, 답으로 보여요.

오른쪽은 호출 기록에 없는 근거입니다. 답에 적힌 조회를 기록에서 찾을 수가 없어요. 모델은 그럴듯한 조회문을 얼마든지 적을 수 있거든요. 실제로 돌려 보지 않았더라도요. 이때 시스템은 그 답을 답으로 보여 주지 않습니다. 답할 수 없음과 그 이유를 보여 주고, 에이전트가 낸 답은 근거가 확인되지 않았다는 표시와 함께 접어 둬요.

실행 기록은 이렇게, 사람이 읽는 데만 쓰이는 게 아니에요. 시스템이 에이전트의 말을 검사하는 데도 쓰입니다. 첫 실습에서 에이전트에게 질문해 보면, 이 표시를 직접 보게 될 거예요.

이 확인 표시의 범위도 구분해야 합니다. 서버가 확인하는 것은 적힌 도구와 조회가 성공한 호출 기록에 대응하는지입니다. 반환된 수량을 답에 정확히 옮겼는지, 조회 기간이 질문에 맞는지까지 자동으로 정답 판정하는 것은 아니에요. 근거가 여러 개라면 각각의 확인 상태도 봅니다. 첫 실습에서도 표시만 보고 끝내지 않고, 실제 반환값과 답의 내용을 대조할 겁니다.

[SLIDE 55] 기준이 지지한 원인과 가설: 판정의 범위

근거를 적을 때 조심할 게 하나 더 있습니다. 두 값이 함께 변했다는 것만으로 원인을 정하면 안 돼요.

화면 왼쪽은 단정하는 경우예요. 유온이 오르는 동안 냉각 효율이 떨어졌습니다. 그래서 쿨러가 원인이라고 적어요. 그런데 팬 구동 불량이나 주변 온도 상승에서도 유온 상승은 나타날 수 있죠. 같은 쿨러 성능 저하 경보에 원인 후보가 셋이었던 게 그래서예요.

오른쪽은 등록된 증거 기준과 비교하는 경우입니다. 냉각 효율은 낮았고, 팬 진동은 이 사례의 기준에서 정상 범위였어요. 캐비닛 온도도 주변 온도 상승을 지지하는 기준에 못 미쳤습니다. 그래서 이 모델에 등록된 후보 중에서는 쿨러 핀 오염이 근거의 지지를 받았어요. 팬 진동이 낮다는 사실만으로 팬의 모든 문제가 없다고 증명한 것은 아닙니다.

화면에서 확인된 원인이라고 부르던 것도 이 판정 범위 안에서 읽어야 합니다. 관측값을 여러 기준과 비교해 가설을 좁힌 것이고, 모든 현장 원인을 확인한 최종 조사와 같지는 않아요. 중요한 조치는 정해진 현장 점검과 승인 절차로 이어져야 합니다.

값을 읽지 못해서, 지우지도 확인하지도 못한 후보가 있다면 어떻게 할까요? 그 후보는 가설이라고 표시합니다. 확인된 원인과 가설을 가르는 것, 이게 에이전트가 근거를 다루는 마지막 장치예요.

이 단원 처음에, 사람이 추천안을 믿고 확인하려면 무엇이 남아야 하느냐고 물었죠. 결과는 정해진 항목으로 내고, 과정은 호출마다 기록으로 남깁니다. 근거는 식별자로 인용해서 기록과 대조하고요. 그리고 확인된 것과 가설을 나눠 적습니다. 이게 다 갖춰져야 사람이 추천안을 확인하고 승인할 수 있어요.

[SLIDE 56~57] 실습: 나의 첫 업무 에이전트 구성하기

모델의 말과 도구의 결과를 구별하고, 근거가 부족할 때 멈추는 기준도 살펴봤습니다. 이 기준을 직접 확인하도록 두 에이전트 실습을 준비했습니다. 첫 실습에서는 업무 에이전트를 하나 구성해 봅니다. 포털의 에이전트 화면에서 이름과 목표와 역할을 적고, 쓸 수 있는 읽기 도구 가운데 필요한 걸 골라요. 두 에이전트의 정의를 견줄 때 봤던 그 항목들이죠.

구성한 에이전트에게는 질문을 하나 던져 볼 겁니다. 답이 나오면 아래 붙은 근거와 도구 호출 기록을 확인해요. 확인 표시가 있는지 보고, 실제 반환값이 답에 맞게 쓰였는지도 대조합니다. 어떤 자료를 읽었는지와 그 자료로 제대로 답했는지를 함께 보는 거죠.

마지막으로 쿨러 사건을 맡은 기준 에이전트와 견줘 보세요. 목표와 역할과 도구가 어떻게 다른지 적으면 돼요. 남는 결과물은 내가 만든 에이전트, 그리고 근거가 붙은 질문 한 건의 기록입니다.

첫 활동이 에이전트의 구성을 익히는 일이라면, 다음 원인 추론 활동은 그 구조를 서로 다른 업무에 적용하는 일입니다.

[SLIDE 58~59] 실습: 설비 이상의 원인 추론과 점검 순서 결정

두 번째 실습에서는 원인을 추론하고 점검 순서를 제안하는 에이전트를 만듭니다. 첫 실습의 포털 구성에서 한 걸음 더 들어가, 바이브 코딩으로 근거를 읽고 결과를 나누는 방식을 직접 구현할 거예요.

화면의 네 칸을 따라가 볼게요. 에이전트는 공용 온톨로지에서 설비, 센서, 고장 유형, SOP로 이어진 관계를 읽어 원인 후보를 얻습니다. 그다음 제공된 센서 입력을 증거 기준에 대어 보고요. 결과는 등록된 기준이 지지하는 원인, 아직 확인하지 못한 가설, 기준에 맞지 않는 후보로 나눠요. 그리고 마지막 칸, 다음에 점검할 걸 순서대로 적습니다.

성공 기준은 확인되지 않은 원인을 단정하지 않고 가설로 표시하는 거예요. 남는 결과물은 점검 계획 파일과, 그 계획에 쓴 근거입니다.

점검 계획에 원인 이름만 쓰는 것이 아니라, 왜 그 점검이 필요한지까지 남기는 활동입니다.

[이론 02] 실전반 4회차

[SLIDE 60] 에이전트 스킬과 SKILL.md

앞에서 구성한 에이전트가 반복해서 따를 수행 방법을 에이전트 스킬로 정리하겠습니다. 여덟 번째 단원, 에이전트 스킬과 그 파일인 SKILL.md부터 시작하겠습니다.

쿨러 성능 저하 경보 한 건을 에이전트가 맡았다고 해 볼게요. 오늘 경보는 이렇게 처리하고, 내일 똑같은 경보는 또 다르게 처리한다면 어떨까요? 믿고 맡기기 어렵겠죠. 같은 종류의 일을 맡길 때마다 필요한 절차를 빠뜨리지 않게 하려면, 무엇을 어디에 적어 두어야 할까요?

[SLIDE 61] 목표와 도구만 받은 에이전트가 정해야 하는 것

일단 에이전트에게 목표와 도구만 줬다고 해 보겠습니다. 목표는 경보의 원인을 진단하고, 조치 후보를 비교해서, 추천안을 올리는 거예요. 그리고 에이전트 앞에는 판단 엔진과 업무 DB의 도구가 합쳐 서른 개 넘게 놓여 있습니다. 정확한 수는 첫 카드에 적어 뒀어요.

이 상태라면 에이전트가 스스로 정해야 하는 게 화면의 카드 네 장이에요.

첫 카드, 무엇부터 부를까요? 그 많은 도구 가운데 첫 호출을 골라야 합니다. 원인 진단부터 할지, 생산 오더부터 볼지, 시계열 값부터 읽을지, 고를 게 많죠.

두 번째 카드, 어떤 순서로 부를까요? 원인을 모르는 채로 조치 후보를 비교할 수는 없잖아요. 순서가 뒤바뀌면 판단에 들어가야 할 근거가 빠집니다.

세 번째 카드, 근거가 없을 때는 어떻게 할까요? 값을 읽지 못했는데 원인을 하나 골라 버리면, 그건 판단이 아니라 지어낸 답이죠.

마지막 카드, 무엇을 내고 끝낼까요? 화면에서 강조된 카드가 이 자리예요. 결과의 모양이 실행마다 다르면, 그 결과를 받아서 사람에게 승인을 받는 다음 단계가 값을 찾지 못합니다.

물론 에이전트가 매번 그럴듯하게 정할 수는 있어요. 문제는 같은 경보인데도 실행마다 과정과 결과의 모양이 달라질 수 있다는 겁니다. 업무에서는 결과가 맞는 것만큼, 같은 방법으로 했다는 걸 확인할 수 있어야 하거든요.

[SLIDE 62] 수행 방법을 지침으로 정해 실행 앞에 둔다

사실 사람의 작업에서는 이 문제를 이미 풀어 봤습니다. 작업 순서와 조건을 SOP로 적어 두고, 작업자가 그 문서를 보면서 일하게 했죠. 에이전트에게도 같은 방법을 씁니다.

그림을 볼게요. 왼쪽에서 경보가 들어오고, 그 옆이 에이전트입니다. 에이전트는 오른쪽의 지식 그래프, 시계열 DB, 업무 DB에 도구를 불러서 값을 받아요. 받은 결과를 보고 다음 호출을 정하고, 이 고리를 돌다가 아래쪽의 추천안을 내고 끝납니다.

그럼 뭐가 달라졌을까요? 맨 위에 놓인 문서 하나입니다. 부를 도구의 순서, 근거가 없을 때의 행동, 결과의 모양을 여기에 적어 두고, 에이전트가 일을 시작할 때 이걸 먼저 읽게 하는 거예요. 실제 기록을 봐도 그렇습니다. 쿨러 경보의 처리 건이 열리자, 에이전트가 가장 먼저 한 일은 이 지침 파일을 읽는 거였어요. 그때 남긴 말이 「먼저 배정된 스킬 절차를 읽고, 필요한 판단 도구를 불러오겠습니다」였습니다. 첫 도구인 원인 진단은 그 뒤에야 불렀고요.

그러니까 실행 고리 자체는 그대로예요. 돌아온 값을 읽고 그 뜻을 해석하는 일은 여전히 에이전트가 합니다. 앞부분에서 지침은 뼈대라고 했던 게 이 뜻이죠. 달라진 건, 앞 장의 카드 네 장에 대한 답을 에이전트가 실행마다 새로 정하지 않아도 된다는 거예요. 그 고리를 어떤 순서로 돌고, 언제 멈추고, 무엇을 내고 끝낼지 기준을 주는 겁니다. 다만 읽는 지침이 생겼다고 모든 실행이 강제로 같아지는 것은 아니에요. 실제 호출과 결과가 지침을 지켰는지는 실행 기록으로 확인해야 합니다.

[SLIDE 63] 에이전트 스킬: 이름을 붙여 재사용하는 업무 지침

이렇게 적어 둔 지침을 에이전트 스킬이라고 부릅니다. 화면 첫 줄의 정의를 보세요. 특정한 일을 수행하는 방법을, 이름을 붙여 적어 둔 지침 묶음이에요.

앞부분에서 에이전트 하나를 정하는 항목으로 목표, 도구, 업무 지침 셋을 봤습니다. 그때 업무 지침은 에이전트 스킬이라는 파일로 준다고만 하고 넘어갔죠. 이번 시간에 그 파일을 제대로 열어 봅니다.

이 정의에서 중요한 말은「이름」입니다. 이름이 있어야 같은 일이 다시 왔을 때 그 지침을 꺼내 쓸 수 있거든요. 한 번 적고 버리는 메모가 아니라, 여러 실행에 되풀이해서 쓰는 지침이라는 뜻이죠.

둘째 줄은 파일입니다. 중심 파일은 SKILL.md, 스킬 문서입니다. 스킬 이름과 같은 폴더 안에 놓이고, 이름과 설명을 적은 머리말, 수행 방법을 적은 본문으로 이루어져요. 필요하면 참고 문서나 실행 스크립트를 같은 묶음에 둘 수도 있습니다. 이 수업의 HYD 사례에서는 중심 지침 파일부터 읽겠습니다.

셋째 줄이 실제 예예요. 쿨러 성능 저하 경보에 쓰는 HYD의 스킬은 냉각 긴급 대응이라는 뜻의 이름을 갖고 있습니다. 경보가 다시 오면 에이전트는 이 이름으로 같은 지침을 꺼내 읽습니다.

[SLIDE 64] 스킬이 놓이는 자리: 작업 폴더의 스킬 폴더

이 파일은 실제로 어디에 놓일까요? HYD에서는 에이전트가 실행할 때마다 작업 폴더를 하나씩 받습니다. 화면 왼쪽이 그 폴더의 모양이에요.

맨 위 입력 폴더부터 볼게요. 여기에는 이번에 맡은 일의 입력 파일이 있습니다. 어느 설비에서 어떤 경보가 났는지가 여기에 들어 있죠.

가운데가 스킬 폴더예요. 배정된 스킬이 폴더째 놓이는데, 폴더 이름이 곧 스킬 이름입니다. 그 안에 SKILL.md가 있고요.

맨 아래 출력 폴더에는 에이전트가 결과를 써 넣을 파일 자리가 미리 정해져 있습니다. 앞부분에서 본 구조화 출력, 약속한 모양의 JSON이 바로 이 파일에 담겨요. 자리가 하나로 정해져 있으니까, 시스템은 늘 같은 곳에서 결과를 꺼내 형식이 맞는지 검사할 수 있죠.

쿨러 경보의 실행 기록을 봐도, 첫 줄에 배정된 스킬 하나를 작업 폴더에 넣고 실행한다고 적혀 있어요. 스킬 본문은 평소에는 포털의 데이터베이스에 보관돼 있다가, 실행이 시작될 때마다 이 자리에 파일로 써지는 거예요. 에이전트에게 주는 지시문에도 배정된 스킬의 이름과 파일 경로를 알려 주는 절이 붙고요. 이렇게 입력, 지침, 결과의 자리를 알려 주면 읽고 쓸 대상을 찾기 쉬워집니다. 화면의 폴더 경로는 이 실행 환경이 택한 배치예요. 다른 제품에서는 경로가 달라도, 이번 일의 입력과 재사용할 지침과 남길 결과를 구분한다는 원리는 같습니다.

[SLIDE 65] SKILL.md 원문 ①: 머리말과 요약

이제 에이전트가 실제로 읽은 SKILL.md를 열어 보겠습니다. 화면 왼쪽이 그 앞부분이에요. 맨 위에 구분선 두 줄이 보이죠? 그 사이가 머리말입니다. 이름과 설명, 두 항목이 들어 있어요.

이름은 폴더 이름과 같은 스킬의 이름이고요, 설명은 어떤 경우에 쓰는 스킬인지를 한 줄로 적은 겁니다. 원문에는 쿨러 과열 경보에서 원인을 진단하고 냉각 조치 카드를 낸다고 적혀 있어요. 여기서 쿨러 과열 경보는, 우리가 지금까지 불러 온 쿨러 성능 저하 경보와 같은 경보입니다.

머리말 아래로 내려가면 본문이 요약 문단으로 시작해요. 경보 한 건에서 원인을 진단하고, 냉각 조치 후보를 비교해서, 추천 카드 한 장과 지는 대안을 올린다고 했죠. 여기서 말하는 추천 카드가 바로 추천안입니다.

마지막 두 줄은 꼭 짚고 갈게요. 승인 전에는 조회와 계산만 하고, 설비 명령은 사람이 승인한 뒤 시스템이 낸다고 적혀 있습니다. 스킬이 맡는 일의 범위와 그 한계를 처음부터 밝혀 둔 거예요. 에이전트가 할 일만큼, 하지 않을 일을 적어 두는 것도 지침의 몫입니다.

[SLIDE 66] 머리말의 name과 description: 스킬을 고르는 기준

머리말의 이름과 설명은 역할이 다릅니다. 이름은 어떤 스킬인지를 구별하는 식별자예요. 설명은 어떤 일에 이 스킬이 필요한지를 판단할 단서입니다.

이름은 파일과 지침 목록을 서로 연결합니다. 화면처럼 소문자와 숫자, 하이픈을 쓰고 폴더 이름과 맞춥니다. HYD의 저장 검사는 길이를 64자 이내로 제한해요. 이 규칙을 외우는 것보다, 목록에서 고른 이름이 실제로 읽을 파일과 일치해야 한다는 이유를 이해하면 됩니다.

설명에는 언제 쓰고 무엇을 내는지를 함께 적습니다. 그냥 경보 처리라고만 적으면 쿨러 경보인지 정기 정비 경보인지 가리기 어렵죠. 쿨러 경보의 원인을 진단하고 냉각 조치 카드를 낸다고 쓰면, 맡길 일과 기대하는 결과가 함께 드러납니다.

스킬을 많이 갖춘 환경에서는 먼저 이런 짧은 이름과 설명을 보여 주고, 필요한 스킬의 본문을 나중에 읽는 방식을 쓸 수 있어요. 모든 지침을 매번 한꺼번에 읽으면 관련 없는 내용까지 들어오고, 지금 필요한 절차를 찾기 어려워지기 때문입니다.

HYD에서는 그 전에 배정 과정이 하나 더 있습니다. 에이전트나 업무 단계에 지정된 스킬을 골라 작업 폴더에 넣고, 그 목록을 실행 지시문에 알려 줘요. 설명 한 줄을 적었다고 모든 에이전트가 자동으로 그 스킬을 찾아 쓰는 것은 아닙니다. 사용할 수 있게 배정하는 일과, 배정된 지침 중 지금 일에 맞는 것을 읽는 일을 구분해 두세요.

[SLIDE 67] SKILL.md 원문 ②: 절차

같은 파일에서 이번에는 절차 부분입니다. 왼쪽 원문을 보면 단계가 다섯 개 있고, 번호가 권장하는 수행 순서예요.

1단계는 입력을 읽을 자리를 가리킵니다. 작업 폴더의 입력 파일에서 설비, 경보의 종류, 경보 번호, 사건 번호를 읽으라고 했어요.

2단계는 원인 진단입니다. 부를 도구의 이름과 넘길 값이 그대로 적혀 있죠. 도구 이름 뒤의 줄표도 보세요. 원인 1순위, 고장 유형, 가이드 카드를 얻는다고, 무엇을 돌려받는지까지 적어 뒀습니다. 그리고 2단계 끝에 한 줄이 더 붙어 있어요. 진단 결과가 근거를 확인하지 못했다고 하면, 원인을 지어내지 말고 보류로 내라는 지침입니다.

3단계는 말투가 조금 다릅니다.「필요하면」업무 DB로 납기 오더와 출하 대기 로트를 확인하되,「조회만」하라고 적었어요. 꼭 해야 하는 단계인지, 어디까지 해도 되는지를 낱말로 정해 둔 거죠.

4단계는 조치 후보 비교예요. 괄호 안에 원인과 고장 유형이 다시 나옵니다. 2단계에서 돌려받은 값이 4단계에 넘길 값이 되는 거예요. 앞부분에서 진단 결과의 두 식별자가 대안 평가의 물음에 그대로 들어간 장면을 봤죠. 원문에는 그 연결이 이렇게 적혀 있습니다. 원인을 모르면 조치 후보를 비교할 수 없다고 했던 게 바로 이 뜻이고요. 그 아래 줄에는, 추천 안이 왜 이기고 다른 안이 어디서 지는지를 한 줄씩 정리하라고 적었어요.

5단계는 판단 제출입니다. 끝에 돌려받은 번호가 판단 번호라고 적어서, 결과에 낼 값이 어디서 오는지도 정해 뒀어요.

이렇게 짚고 보면, 도구 이름이 그대로 적힌 건 2단계, 4단계, 5단계입니다. 1단계는 읽을 자리를, 3단계는 범위를 정하고요. 그러니까 에이전트는 그 많은 도구 가운데 무엇을 어떤 순서로 부를지 추측하지 않아도 됩니다. 절차가 순서와 단계마다의 범위를 알려 주거든요. 필요한 값이 다음 단계로 제대로 넘어갔는지 확인할 기준도 여기서 얻습니다.

[SLIDE 68] SKILL.md가 정하는 여섯 가지

지금까지 원문에서 본 걸 여섯 가지로 묶어 보겠습니다. 왼쪽은 SKILL.md 한 쪽을 그린 그림이고, 오른쪽은 각 구획이 맡는 일이에요.

위에서부터 갑니다. 맨 위 머리말의 설명이 호출 조건을 맡아요. 언제 쓰는 스킬인지를 정하죠. 그 아래 본문의 첫 문단은 작업 목적을 밝힙니다. 경보 한 건에서 무엇을 내는 일인지요.

가운데 절차에는 세 가지가 들어 있어요. 1단계가 입력을 읽을 자리를 가리키고요, 2단계 끝의 문장이 예외 지침을 맡고, 단계 번호가 호출의 순서가 됩니다.

맨 아래 결과 절이 출력을 정해요. 내놓을 항목의 이름을 여기에 적습니다.

그런데 기억해 둘 게 있어요. 실제 파일에는「호출 조건」이라는 절이나「예외」라는 절이 따로 있지 않습니다. 설명 한 줄과 절차 안의 한 문장이 그 일을 맡을 뿐이에요. 그러니까 여섯 가지는 제목이 붙은 칸이 아니라, 스킬 한 편이 빠짐없이 답해야 하는 물음입니다. 언제, 무엇을 위해 쓰고, 무엇을 받고, 막히면 어떻게 하고, 어떤 순서로 해서, 무엇을 내는가. 나중에 직접 스킬을 쓸 때 이 물음에 답이 다 있는지 확인하면 됩니다.

[SLIDE 69] 예외 지침: 근거가 없을 때 지어내지 않는다

여섯 가지 가운데 예외 지침이 실제로 쓰이는 장면을 보겠습니다. 원인 진단 도구의 응답에는 근거를 확인했는지 알려 주는 항목이 하나 있어요. 화면에 withheld라고 적힌 항목입니다.

왼쪽 칸이 근거가 확인된 응답이에요. 보류하지 않는다는 표시가 오고, 원인 1순위가 정해집니다. 쿨러 사건에서는 쿨러 핀 오염이었죠. 둘째 줄을 보면, 이때 응답에는 관측값이 허용된 나이 안에 있다는 신선도 표시도 함께 왔어요. 그러면 에이전트는 절차의 다음 단계로 넘어가요.

오른쪽 칸은 근거가 확인되지 않은 응답입니다. 판단을 보류한다는 표시가 오는 경우죠. 값이 오래됐거나, 조회한 증거가 어떤 원인과도 맞지 않을 때예요. 값이 오래됐다는 건, 앞부분에서 본 신선도가 허용 범위를 벗어난 경우고요. 이럴 때 스킬은 원인을 고르지 말고 보류로 내라고 지시합니다. 보류로 낼 때는 앞부분에서 본 보류의 형식대로, 상태와 이유와 증거를 담아 내죠.

왜 이 문장을 굳이 적어 둬야 할까요? 에이전트는 어떻게든 답을 만들어 내는 쪽으로 움직이기 쉽거든요. 근거가 없는데도 제일 그럴듯한 원인을 골라 다음 단계로 가 버리면, 그 뒤의 조치 비교와 추천이 전부 잘못된 출발점 위에 쌓입니다. 막혔을 때의 행동을 미리 정해 둔다는 건, 보류도 하나의 정상적인 결과로 인정한다는 뜻이에요.

[SLIDE 70] 결과 절이 정한 항목과 실제 출력

이번에는 결과입니다. 왼쪽 위가 SKILL.md의 결과 절이고, 그 아래가 에이전트가 실제로 쓴 결과 파일이에요. 결과 절이 이름을 정해 둔 다섯 항목이 파일에 그대로 들어 있는 게 보이죠?

처음 두 항목은 원인과 고장 유형입니다. 지식 그래프의 식별자를 그대로 냈어요. 쿨러 핀 오염, 그리고 쿨러 냉각 성능 상실을 가리키는 값이에요.

가운데 판단 항목에는 추천한 조치와 설명, 후보의 순위가 들어 있습니다. 추천 값은 팬 최대 운전을 가리키는 식별자고요. 이름이 skill로 시작하지만, 지금 배우는 에이전트 스킬이 아니라 조치 방법을 가리키는 값이에요. 둘은 잠시 뒤에 가릅니다. 설명에 무엇을 쓸지도 스킬 원문의 결과 절이 정해 뒀어요. 원인, 추천한 안, 그리고 다른 안이 왜 지는지를 두세 문장으로 쓰라고 했습니다. 절차 4단계에서 한 줄씩 정리하라던 그 까닭이 여기에 들어가는 거예요. 승인하는 사람이 고른 안만 보는 게 아니라, 고르지 않은 안의 까닭까지 보게 하려는 거죠.

마지막 항목은 제출한 판단에 붙은 번호예요. 그런데 쿨러 경보는 같은 스킬로 두 번 처리됐고, 이 파일은 그 둘째 건의 결과입니다. 그래서 번호도 둘째 건의 것이에요. 두 실행은 조금 뒤에 나란히 봅니다.

결과 절이 이름을 굳이 정해 두는 까닭은 받는 쪽에 있습니다. 결과를 받아서 사람에게 승인을 받는 다음 단계는, 값을 항목의 이름으로 찾아요. 이름이 실행마다 바뀌면 다음 단계가 값을 못 찾겠죠. 결과 절은 에이전트의 일과 그다음 사람의 일을 잇는 약속인 셈입니다.

이렇게 언제 쓰는지부터 무엇을 내는지까지 SKILL.md 한 편에 적어서 실행 앞에 두면, 같은 종류의 경보가 다시 와도 참조할 절차와 결과 형식을 다시 사용할 수 있습니다.

[SLIDE 71] 재사용 지침: SOP · 프롬프트와의 구분

스킬 한 편이 무엇으로 이루어지는지 원문으로 확인했으니, 아홉 번째 단원에서는 스킬을 비슷해 보이는 두 가지와 갈라 보겠습니다. 에이전트 스킬은 절차를 적은 문서죠. 그런데 쿨러 사건에는 이미 절차 문서가 하나 있었습니다. 팬을 최대로 올리는 SOP예요. 같은 사건에 쓰인 이 두 절차는 무엇이 다를까요? 또 방법을 매번 프롬프트로 알려 주는 것과 스킬로 남겨 두는 것은 어떻게 다를까요? 이 두 가지를 가른 다음, 스킬이 실제로 다시 쓰인 모습까지 확인하겠습니다.

[SLIDE 72] 한 사건에 쓰인 두 절차: 에이전트 스킬과 SOP

쿨러 사건 한 건을 시간 순서로 늘어놓았습니다. 왼쪽 끝이 경보가 난 때고요, 가운데를 조금 지난 자리가 운전원이 승인한 때예요. 정확한 시각은 라벨에 적어 뒀는데, 경보에서 승인까지 2분이 채 안 걸렸습니다.

왼쪽 하늘색 구획부터 볼게요. 에이전트 스킬이 이끄는 구간입니다. 아래 줄을 따라가면 SKILL.md, 에이전트, 추천안이 차례로 있죠. 에이전트가 스킬을 읽고, 조회와 계산을 해서 추천안을 냈어요. 모두 승인 전의 일입니다.

운전원이 팬 최대 운전을 승인하고 나면 오른쪽 회색 구획으로 넘어갑니다. 아래 줄의 SOP-COOL-11과 팬 100 %가 여기 있어요. 이 구간에서는 SOP가 설비에 할 일을 정했습니다. 팬 속도를 100 %로 올리는 작업이죠. 그 명령도 에이전트가 아니라, 승인을 받은 시스템이 냈고요.

이렇게 두 절차가 쓰인 때는 사람의 승인을 사이에 두고 딱 갈립니다. 승인 전에는 정보를 모으고 판단을 만드는 절차가, 승인 뒤에는 설비를 실제로 바꾸는 절차가 쓰여요. 스킬 원문의 마지막 두 줄, 승인 전에는 조회와 계산만 한다는 문장 기억나시죠? 그 문장이 바로 이 경계를 적어 둔 거였습니다.

[SLIDE 73] SOP와 에이전트 스킬: 읽는 이와 적는 내용

두 문서를 나란히 놓으면, 이 사례에서 누구에게 무엇을 알려 주는지가 분명해집니다. 왼쪽 SOP는 현장 작업자가 참고하는 표준 작업 절차예요. 설비를 다루는 순서와 안전 조건, 금지 사항을 적습니다. 화면의 팬 최대 운전 절차가 그 예입니다.

오른쪽 에이전트 스킬은 AI가 업무를 수행할 때 읽는 지침이에요. HYD의 냉각 대응 스킬에는 어떤 도구로 원인을 확인하고, 근거가 부족하면 어떻게 멈추며, 어떤 항목으로 추천안을 낼지가 적혀 있습니다.

여기서 승인 전후의 구분은 HYD가 이 업무에 정한 역할 배분입니다. SOP라는 말이 반드시 승인 뒤의 설비 조작만 뜻하는 것은 아니에요. 작업 전 안전 확인이나 정보 확인도 SOP에 들어갈 수 있습니다. 에이전트 스킬 역시 일반적으로는 다양한 업무의 지침을 담을 수 있고요.

이 수업에서 지킬 경계는 구체적입니다. 냉각 대응 에이전트는 판단을 만들고 제출하며, 설비 조작은 사람의 승인 뒤 시스템이 맡습니다. 같은 절차 문서라도 누가 읽는지, 무엇을 수행하는지, 실행

권한은 어디에 있는지를 함께 봐야 합니다. 문서에 하라고 썼다는 사실만으로 그 권한이 생기지는 않습니다.

[SLIDE 74] 「Skill」의 두 쓰임: 조치 방법과 에이전트 스킬

여기서 이름 때문에 헷갈리기 쉬운 걸 하나 정리하고 가겠습니다. 지식 그래프에서 조치 방법의 클래스 이름이 Skill이었죠. 앞의 결과 파일에서 본 추천 값도 바로 그 클래스의 식별자였고요.

표의 가운데 열이 조치 방법입니다. SOP 한 편을 지식 그래프로 옮긴 노드예요. 작업 단계, 승인 역할, 근거 매뉴얼과 관계로 이어져 있고, 조치 후보로 추천되고 승인됩니다.

오른쪽 열이 이번 회차에 배우는 에이전트 스킬이에요. 에이전트가 읽는 지침 파일이고, 도구를 부르는 순서와 결과의 모양을 담고 있죠. 판단을 수행할 때 읽힙니다.

이 HYD 스킬 본문은 특정 조치 방법 노드를 직접 가리키지 않아요. 에이전트 스킬 본문에는 팬 최대 운전이라는 조치 방법이 적혀 있지 않습니다. 조치 후보는 도구가 지식 그래프에서 읽어서 돌려줄 뿐이에요. 그래서 이 과정에서는 지식 그래프 쪽을 조치 방법, 지침 파일 쪽을 에이전트 스킬, 이렇게 나눠서 부르겠습니다.

[SLIDE 75] 스킬에 적힌 것과 입력 · 도구에서 얻는 것

방금 스킬 본문에 팬 최대 운전이 적혀 있지 않다고 했죠. 사실 HYD-01이라는 설비 이름도, 쿨러 핀 오염이라는 원인도 안 적혀 있습니다. 그럼 이런 정보는 다 어디서 올까요?

표의 첫 줄이 스킬에 적힌 것입니다. 도구를 부르는 순서, 보류의 조건, 결과 항목의 이름. 그러니까 방법만 있어요.

둘째 줄, 설비와 경보 번호는 작업 폴더의 입력 파일에서 읽습니다. 셋째 줄, 원인과 조치 후보는 도구가 지식 그래프에서 읽어 주고요. 쿨러 핀 오염이라는 원인, 그리고 팬 최대 운전을 비롯한 네 가지 후보가 여기서 옵니다. 넷째 줄의 지금 값, 냉각 효율이 지금 얼마인지, 납기까지 몇 시간 남았는지 같은 값은 도구가 조회하는 그 시점에 가져오죠.

이렇게 나눠 두면 같은 업무 규칙을 적용할 수 있는 다른 사건에도 지침을 다시 쓸 수 있어요. 설비나 경보 번호가 달라졌다는 이유만으로 절차를 다시 쓸 필요가 줄어드는 거죠. 다만 대상 설비가 같은 진단을 지원하는지, 필요한 도구와 규칙이 맞는지는 확인해야 합니다. 앞부분에서 지식과 현재값을 섞지 말라고 했죠. 이번 HYD 스킬은 사건별 원인과 현재 수치를 직접 적어 두지 않습니다. 만약 스킬에 냉각 효율 값을 적어 뒀다면 어떨까요? 다음 경보에서는 그 값이 이미 묵은 값인데도 판단에 들어가겠죠. 사건마다 달라지는 것은 스킬에 적지 않는다, 이게 재사용의 조건이에요.

[SLIDE 76] 같은 스킬, 다른 경보: 두 번의 실행

그 재사용이 실제로 일어난 모습입니다. 왼쪽의 스킬 한 편이, 오른쪽의 서로 다른 쿨러 경보 두 건에 쓰였어요.

오른쪽 위의 첫째 경보와 그 아래 둘째 경보는 번호가 다른 별개의 건입니다. 두 실행 모두 같은 스킬을 읽고 같은 절차를 따랐어요. 그리고 저마다 판단을 하나씩 제출했죠. 판단 번호도 서로 다릅니다.

다만 같은 스킬을 따랐다고 해서 두 실행이 한 글자까지 똑같았던 건 아니에요. 앞부분에서 봤듯이, 둘째 실행은 지침에 없던 조치 효과 예측을 더 불렀죠. 스킬이 제공하는 건 함께 참고할 절차의 뼈대와 결과 형식입니다. 실제로 지켰는지는 두 실행의 호출과 출력으로 따로 확인합니다.

맨 아래 흐린 줄은 아직 오지 않은 경보예요. 같은 종류의 경보가 또 오면, 같은 스킬이 또 쓰입니다.

한번 정리한 지침을 여러 실행이 함께 쓸 수 있으니, 매번 설명할 수고가 줄어듭니다. 그런데 반대도 마찬가지예요. 지침에 잘못이 있으면 그 잘못도 모든 실행에 따라갑니다. 그래서 스킬은 한 번 쓰고 끝내는 게 아니라, 실행 기록과 대조하면서 고쳐 가는 자산이에요.

[SLIDE 77] 한 번의 프롬프트와 재사용하는 지침

방법을 그때그때 프롬프트에 적어 주는 방식과, 이름을 붙여 관리하는 스킬을 비교해 보겠습니다. 둘 다 에이전트에게 읽힐 수 있는 지침이에요. 차이는 반복해서 쓸 방법을 어떻게 보관하고 적용하느냐에 있습니다.

왼쪽은 이번 요청에만 필요한 조건을 직접 적는 경우입니다. 오늘 받은 회의록 한 편을 정리해 달라는 요청처럼, 입력과 목적이 이번 일에 묶여 있죠. 같은 절차를 매번 복사해서 붙이면 수정한 판과 옛 판이 섞이거나 중요한 예외가 빠질 수 있습니다.

오른쪽은 반복되는 절차를 스킬로 관리하는 경우예요. 회의록이 바뀌어도 담당자와 기한이 없으면 미정으로 남긴다는 규칙은 같죠. 그 공통 방법을 이름 있는 지침으로 보관하고, 요청에는 이번 회의록과 추가 조건을 주면 됩니다.

프롬프트도 템플릿으로 저장해서 다시 쓸 수 있습니다. 스킬만 재사용할 수 있다는 뜻은 아니에요. 스킬의 가치는 관련 지침과 자료를 한 묶음으로 관리하고, 필요한 업무에서 찾아 쓰도록 연결하는 데 있습니다.

HYD에서는 보관된 스킬을 실행 준비 때 작업 폴더에 써 줍니다. 보관본을 수정하면 이후 준비되는 실행에서 새 내용을 받는 방식이에요. 이미 진행 중인 모든 실행이 즉시 바뀐다고 생각하면 안 됩니다. 어떤 실행에 어느 지침이 제공됐는지 남겨야, 수정 전후 결과를 비교할 수 있습니다.

[SLIDE 78] 같은 틀의 세 스킬과 그 스킬을 붙인 에이전트

이렇게 이름을 붙여 남긴 HYD의 기본 시나리오 스킬 세 개를 보면, 저마다 에이전트 하나에 붙어 있습니다. 표의 첫 칸이 그 에이전트예요. 쿨러 경보를 맡는 냉각 긴급 대응, 정기 정비 시기를 정하는 정기 정비 계획, 예비품 발주를 맡는 예비품 구매입니다.

가운데 칸에는 각 스킬의 이름과 설명이 있어요. 둘째 줄의 설명을 위에서부터 이어 읽어 볼게요. 쿨러 과열 경보에는 원인 진단과 냉각 조치 카드, 정기 정비가 돌아오면 언제 정비할지 카드, 예비품 재고가 기준을 벗어나면 발주 카드. 셋 다 앞에서 본 설명의 꼴 그대로, 무엇이 오면 쓰고 무엇을 내는지를 한 줄에 담았죠. 세 스킬 모두 머리말, 절차, 결과로 이루어져 있어서 틀도 같습니다. 다른 건 절차에 적힌 업무, 그리고 그 절차가 부르는 도구예요.

오른쪽 칸은 각 에이전트에 연결한 서버입니다. 스킬의 절차가 부를 도구가 있는 곳이죠. 여기서 서버는 일단, 도구를 모아서 내주는 프로그램이라고 생각하시면 됩니다. 칸의 첫 줄, 지식 그래프 서버와 판단 엔진 서버는 세 에이전트가 똑같이 연결했어요. 다른 건 둘째 줄의 업무 DB 쪽입니다. 냉각 긴급 대응은 업무 DB 전체에 닿는 서버를, 정기 정비 계획은 정비 쪽 서버를, 예비품 구매는 구매 쪽 서버를 연결했어요. 왜 이렇게 나눴는지는 안전 기준을 다룰 때 다시 보겠습니다.

하나 기억해 둘 게 있어요. 스킬은 만들어 두기만 해서는 쓰이지 않습니다. HYD에서는 에이전트의 배정 목록이나 업무 단계의 지정에 들어 있어야 해당 실행에 제공됩니다. 그래서 새 업무를 맡길 때는 세 가지를 해야 합니다. 같은 틀에 그 업무의 절차를 적고, 그 일을 맡을 에이전트에 붙이고, 절차가 부를 도구를 연결하는 거죠.

[SLIDE 79] 스킬을 쓸 때 지키는 네 가지

이제 스킬을 직접 쓸 때 지킬 것을 네 가지로 줄여 보겠습니다. 실습에서 회의록 스킬을 쓸 때 이 카드 네 장을 그대로 씁니다.

첫 카드, 설명에는 어떤 요청 한 건에서 무엇을 내는 스킬인지를 한 줄로 밝힙니다. 사람이 배정할 때도, 에이전트가 적용할 절차를 찾을 때도 이 설명을 단서로 쓰니까요.

두 번째, 절차의 도구 이름은 에이전트에 연결된 도구의 실제 이름을 그대로 옮겨 적습니다. 그럴듯한 이름을 지어내면, 에이전트는 없는 도구를 찾다가 멈춰요.

세 번째, 근거나 남는 후보가 없을 때 어떻게 할지 정해 둡니다. 지어내지 말고, 없으면 없는 그대로 내게 하는 거예요. 예를 들어 규칙으로 후보를 다 빼고 나니 남는 안이 없다면, 추천을 억지로 만들지 말고 빈 목록과 함께 안마다 빠진 이유를 내게 하는 식이죠.

네 번째, 결과 항목의 이름을 정해 두고 바꾸지 않습니다. 다음 단계가 그 이름으로 값을 읽으니까요.

이 네 가지는 스킬이 SOP나 프롬프트와 다른 바로 그 지점에서 나옵니다. SOP는 사람이 읽지만 스킬은 에이전트가 읽죠. 모호한 문장을 모델이 그럴듯하게 해석하면 의도와 다른 도구를 고를 수 있어요. 재사용되는 지침의 모호함은 다음 실행에도 이어질 수 있고요. 그래서 스킬은 짧더라도 고르는 기준, 부를 이름, 막혔을 때의 행동, 내놓을 이름, 이 네 가지가 분명해야 합니다.

이제 회의록을 작업 목록으로 바꾸는 스킬을 직접 작성하겠습니다. 그 뒤에는 이 지침이 부르는 도구의 연결을 살펴보겠습니다.

[SLIDE 80~81] 실습: 회의 내용을 후속 업무로 바꾸는 스킬 작성

세 번째 실습은 회의록 스킬 작성입니다. 앞에서 배운 스킬의 작성 기준을 적용해, 회의 내용을 후속 업무로 바꾸는 스킬을 직접 써 봅니다. 예제 회의록을 입력으로 받아서, 결정사항, 담당자, 완료 시점을 뽑아내는 SKILL.md예요. 앞에서 카드 네 장으로 정리한, 스킬을 쓸 때 지킬 네 가지를 그대로 쓰면 됩니다. 언제 쓰는 스킬인지 설명 한 줄로 밝히고, 절차와 결과 항목의 이름을 정하세요.

예외 지침도 꼭 넣습니다. 담당자나 기한이 회의록에 없으면, 지어내지 말고「미정」으로 표시하게 하는 거예요. 그 스킬로 Markdown과 JSON 작업 목록을 만들면, 그 목록이 산출물로 남습니다. 성공 기준은 넷째 카드에 적었어요. 요약에서 끝나지 않고 누가 언제까지 무엇을 하는지가 항목마다 남고, 항목마다 회의록 원문의 근거 문장이 있으면 성공입니다.

같은 안건이 반복되거나 표현이 다르다고 함부로 합치지 말고, 같은 일인지 판단한 근거와 불확실한 부분을 남깁니다. 작업 목록을 만드는 것과 실제 담당자에게 업무를 발송하는 것도 구분하세요. 이번 활동은 목록을 만들고 원문과 대조하는 데까지입니다.

[이론 03] 실전반 4회차

[SLIDE 82] MCP의 구성: 호스트 · 클라이언트 · 서버 · 도구

이번 시간에는 스킬이 부르는 도구 쪽을 다룹니다. 도구를 잇는 표준인 MCP의 구성부터, 도구를 찾아 부르는 과정과 연결 방식, 그리고 한 실행 안에서 스킬과 MCP가 나눠 맡는 몫까지 봅니다.

앞에서 본 에이전트 스킬에는 부를 도구의 이름이 적혀 있었죠. 그런데 이름을 적었다고 해서 도구가 실행되지는 않아요. 그 도구는 어디에 있고, 에이전트는 어떤 약속을 따라서 그걸 부를까요? 열 번째 단원에서는 그 약속, MCP가 무엇으로 이루어져 있는지 보겠습니다.

[SLIDE 83] 스킬의 절차가 가리키는 것: 스킬 밖의 도구

스킬 절차의 2단계 한 줄을 화면 가운데에 크게 띄웠습니다. 이 한 줄 안에 이름이 세 개 들어 있어요.

맨 앞은 도구를 내주는 프로그램의 이름입니다. 판단 엔진을 앞에 두고 도구를 내주는 프로그램이에요. 원인 진단은 실제로 여기서 실행됩니다.

가운데는 그 프로그램이 내주는 도구, 원인 진단의 이름이고요. 괄호 안은 도구에 넘길 값의 이름이에요. 설비, 경보의 종류, 경보 번호죠. 이 세 값은 바로 앞의 1단계에서 입력 파일로 읽은 것들입니다. 입력에서 읽은 걸 그대로 도구에 넘기는 거예요.

이렇게 세 이름이 저마다 하나씩 답하고 있습니다. 어느 프로그램에 보낼지, 무엇을 실행할지, 무엇에 대해 실행할지요. 결국 스킬은 이 이름들을 적어 둔 글일 뿐입니다. 원인을 진단하는 계산은 스킬 안에 없어요. 스킬 밖의 프로그램에서 일어납니다.

[SLIDE 84] 시스템마다 따로 잇는 방식과 표준을 거치는 방식

바깥 시스템과 연결하는 방법이 제각각이면 어떤 일이 생길까요? 화면 왼쪽에는 애플리케이션 셋과 시스템 셋이 있습니다. 각각의 조합마다 별도 규격에 맞춘 코드를 만들면, 관리할 조합은 아홉 가지가 됩니다.

오른쪽은 양쪽이 공통 규약을 따르는 방식입니다. 애플리케이션 쪽은 공통 규약으로 요청을 보내고 결과를 읽는 기능을 갖춥니다. 시스템 쪽은 자기 기능을 그 규약에 맞게 내주는 서버를 마련하고요. 화면의 셋 더하기 셋은 이렇게 표준에 맞춰 구현할 쪽을 세는 개념도입니다.

이 숫자를 실제 연결선이나 접속 횟수로 읽으면 안 됩니다. 애플리케이션 하나가 서버 셋을 쓰면 서버마다 연결이 필요합니다. 또 가운데 파란 띠는 모든 요청이 지나가는 중앙 서버가 아니라, 양쪽이 함께 따르는 약속을 나타냅니다.

왜 이런 약속이 도움이 될까요? 시스템 쪽에서 기능을 어떻게 공개하는지 일정해지면, 애플리케이션마다 같은 연결 작업을 처음부터 반복할 일이 줄어듭니다. 반대로 업무 시스템의 데이터 구조와 권한은 여전히 다르니, 그 차이는 각 서버가 맞춰 줘야 합니다. 표준이 줄이는 것은 반복되는 연결 형식의 차이이고, 업무의 의미까지 자동으로 같게 만드는 것은 아니에요.

[SLIDE 85] MCP: AI 애플리케이션과 외부 도구를 잇는 개방형 프로토콜

그 표준이 MCP, Model Context Protocol입니다. 화면 첫 줄의 정의를 볼게요. AI 애플리케이션이 외부의 도구와 데이터에 연결하는 방식을 하나로 정한 개방형 프로토콜이에요.

개방형이라는 말은, 누구나 이 규약에 맞춰 만들 수 있다는 뜻입니다. 그래서 서로 다른 곳에서 만든 애플리케이션과 서버도 공통 규약을 바탕으로 연결할 수 있습니다. 실제 연결에는 지원하는 규약의 판, 연결 방식, 필요한 기능과 인증 조건이 맞아야 해요. HYD에도 그런 예가 있어요. 지식 그래프 서버는 HYD가 만든 게 아니라, 바깥에 공개된 서버를 그대로 가져다 붙인 겁니다. 판단 엔진 서버와 업무 DB 서버는 HYD가 직접 만들었고요. 만든 곳은 달라도, 에이전트 쪽에서는 똑같은 방식으로 연결합니다.

둘째 줄은 메시지 형식이에요. MCP는 JSON-RPC 2.0이라는 형식으로 메시지를 주고받습니다. 원격으로 기능을 부르는 약속을 JSON으로 적은 형식이죠. 요청에는 부를 기능의 이름과 넘길 값이 담기고, 응답에는 결과나 오류가 담겨요.

셋째 줄은 비유입니다. 규격이 같은 단자를 떠올려 보세요. 만든 회사가 달라도 규격이 같으면 꽂히죠. 그런데 다른 점도 있어요. MCP는 연결한 다음에 쓸 수 있는 기능을 맞추고, 서버가 무엇을 내주는지 물어서 알아내는 절차가 있습니다. 이 절차가 뒤에서 볼 도구 목록의 발견이에요.

[SLIDE 86] MCP의 네 구성요소: 호스트 · 클라이언트 · 서버 · 도구

이번에는 MCP로 도구를 호출하는 데 필요한 네 역할을 보겠습니다. 호스트, 클라이언트, 서버, 도구. 그림 왼쪽부터 하나씩 볼게요.

왼쪽의 큰 하늘색 구획이 호스트입니다. 라벨에 적은 대로, HYD에서는 에이전트 프로그램이 이 자리에 있어요. 그 안에 모델과 클라이언트가 들어 있고, 모델은 LLM입니다.

호스트 안에 작은 상자가 세 개 있죠? 이게 클라이언트입니다. 클라이언트 하나는 서버 하나와 일대일로 연결돼요. 서버가 셋이니까 클라이언트도 셋입니다.

가운데 흰 상자들이 서버예요. 위에서부터 지식 그래프 서버, 판단 엔진 서버, 업무 DB 서버입니다. 서버는 오른쪽 끝에 있는 시스템의 기능을 도구로 내줘요. 상자 안의 톱니바퀴가 바로 도구고요.

그림은 가로로 한 줄씩 읽으면 됩니다. 맨 윗줄은 클라이언트 하나, 지식 그래프 서버, 그 뒤의 지식 그래프가 한 줄로 이어져 있죠. 가운뎃줄은 판단 엔진까지, 아랫줄은 업무 DB까지 같은 모양이에요. 이 그림은 각 서버의 주된 대상을 한 줄씩 놓은 것입니다. 서버 하나가 여러 시스템의 값을 모아 주는 구성도 가능합니다.

그래서 에이전트마다 연결할 서버를 고르면, 클라이언트도 딱 그 서버 수만큼만 생깁니다. 서버를 하나 빼면 그 줄이 통째로 사라지는 거죠.

[SLIDE 87] 네 구성요소의 정의와 HYD의 대응

그림으로 자리를 봤으니, 네 구성요소의 정의를 정리하고 HYD에서는 그 자리에 무엇이 있는지 짝지어 보겠습니다. 정의는 화면 가운데 칸의 첫 줄에, HYD의 대응은 둘째 줄에 적어 뒀어요.

첫 줄, 호스트는 모델을 가지고 사용자의 일을 수행하는 애플리케이션이고, 클라이언트를 만들고 관리합니다. HYD에서는 처리 건마다 띄워지는 에이전트 프로그램이 이 자리에 있어요. 모델을 부르고, 연결할 서버마다 클라이언트를 두는 쪽이죠. 이 실행 사례에서는 작업 폴더를 마련하고 프로그램을 띄우는 워커와, 그 안에서 모델과 MCP 연결을 관리하는 호스트를 구분합니다.

둘째 줄, 클라이언트입니다. HYD에서는 실행할 때마다 작업 폴더에 연결할 서버의 설정 파일이 놓이고요, 호스트가 거기 적힌 서버마다 클라이언트를 하나씩 둡니다. 그래서 그 에이전트에 연결한 서버 수만큼 생기는 거예요.

셋째 줄, 서버는 지식 그래프, 판단 엔진, 업무 DB 앞에 놓인 프로그램들입니다.

넷째 줄, 도구는 서버가 내주는, 호출할 수 있는 기능이에요. 이름과 설명과 입력 스키마를 갖고 있습니다. 원인 진단, 조치 후보 비교, 생산 오더 조회 같은 것들이죠.

HYD에는 클라이언트를 가진 쪽이 하나 더 있어요. 포털의 도구 시험 화면입니다. 브라우저 화면의 요청을 받아 포털 백엔드가 MCP 클라이언트로 서버에 연결하고 목록과 결과를 가져옵니다. 실습에서 서버를 등록하고 시험해 볼 때 쓰는 곳이 바로 이 화면입니다.

서버는 도구 외에도 읽을 자료나 재사용할 프롬프트를 제공할 수 있습니다. 이번 HYD 사례에서는 호출 가능한 도구를 중심으로 보는 거예요. 그리고 모델과 호스트는 꼭 구별해 두세요. 모델은 무엇을 부를지 고르는 쪽이고, 호스트는 그 선택을 받아서 실제 연결을 관리하는 애플리케이션이에요. 이 구별은 뒤에서 안전 기준을 다룰 때 다시 씁니다.

[SLIDE 88] HYD의 MCP 서버 다섯

이 교재가 확인한 HYD의 기본 등록 MCP 서버는 다섯 개입니다. 표를 보면 서버마다 닿는 시스템과 내주는 도구가 다르죠.

첫 줄은 지식 그래프 서버예요. 도구 가운데 그래프의 구조를 읽는 것과 조회문을 실행하는 것, 딱 이 둘만 허용해 뒀습니다.

둘째 줄, 강조된 게 판단 엔진 서버입니다. 원인 진단, 조치 후보 비교, 시계열 조회 같은 도구 19개를 내줘요. 쿨러 사건에서 가장 많이 불린 서버죠. 닿는 시스템 칸을 보면 판단 엔진과 시계열 DB가 같이 적혀 있어요. 판단에 필요한 측정값까지 이 서버가 직접 읽기 때문에, 시계열 조회 도구가 여기에 있는 거예요.

셋째 줄은 업무 DB 전체를 내주는 서버로, 도구가 14개예요. 넷째와 다섯째 줄은 업무 DB 가운데 정비 쪽과 구매 쪽만 내주는 서버고요. 도구가 각각 네 개, 세 개입니다. 정비 쪽에는 정비 이력과 정기 정비 상태 같은 도구가, 구매 쪽에는 예비품 재고와 부품 견적 같은 도구가 들어 있어요.

쿨러 경보를 맡은 에이전트가 쓴 건 위의 세 서버예요. 스킬 2단계의 원인 진단도 이 가운데 판단 엔진 서버에 있습니다. 이렇게 스킬에 적힌 도구는 이 서버들 가운데 어딘가에 있고, 호스트는 서버마다 둔 클라이언트를 거쳐서, MCP의 약속대로 그 도구를 부릅니다.

[SLIDE 89] 도구의 발견 · 호출 · 연결 방식

네 구성요소가 무엇인지 알았으니, 열한 번째 단원에서는 그 사이로 메시지가 어떻게 오가는지 따라가 보겠습니다. 호스트는 서버에 어떤 도구가 있는지 어떻게 알까요? 도구를 부를 때는 무엇을 넘기고, 무엇을 돌려받을까요? 그리고 클라이언트와 서버는 어떤 길로 이어질까요?

[SLIDE 90] 도구 목록의 발견과 호출: tools/list와 tools/call

도구를 쓰기까지의 과정을 왼쪽부터 세 단계로 보겠습니다. 첫째는 연결을 시작하는 초기화, 둘째는 도구를 알아보는 발견, 셋째는 실제로 도구를 부르는 호출입니다. 각 단계에서 클라이언트와 서버가 주고받는 내용이 달라요.

첫 단계에서는 초기화 요청을 주고받으며 규약의 판과 지원 기능을 확인합니다. 서버가 도구를 제공하는지도 이때 알려 줘요. 초기화가 끝났다는 알림까지 보내면 정상적인 요청을 주고받을 준비가 됩니다. 서로 지원하는 규약 판이 맞아야 그다음 요청을 주고받을 수 있는 거죠.

그다음은 도구 목록 요청입니다. 이름과 설명, 입력 스키마를 받아 어떤 기능에 무엇을 넘겨야 하는지 알아봅니다. 목록이 크면 나눠 받아 이어 읽을 수도 있어요. 여기서는 도구 목록을 요청하는 표준 이름인 tools/list를 함께 기억해 두면 됩니다.

필요한 도구를 고른 다음에는 도구 이름과 입력 값을 담아 호출합니다. 이때의 표준 이름이 tools/call이에요. 목록을 알아보는 요청과 업무 기능을 실행하는 요청이 서로 다릅니다.

HYD 포털의 연결 검사는 초기화하고 목록을 실제로 받아 보는 데까지 합니다. 그래서 통과했다는 말은 그 설정으로 대화하고 목록을 읽었다는 뜻이에요. 특정 도구에 업무 입력을 넣었을 때 성공하는지, 결과가 업무상 옳은지는 다음 호출에서 확인해야 합니다. 목록 조회가 모든 호출 직전의 필수 규칙인 것은 아니지만, 지금 사용할 도구의 이름과 입력 형식을 아는 데 필요한 발견 과정입니다.

[SLIDE 91] 도구 목록을 연결할 때 묻는 까닭

그런데 굳이 연결할 때마다 서버에 목록을 물어야 할까요? 호스트에 도구를 미리 적어 두면 더 간단해 보이는데 말이죠.

왼쪽처럼 도구를 호스트에 미리 적어 두면 어떻게 될까요? 서버의 도구가 바뀔 때마다 호스트도 고쳐야 합니다. 고치는 걸 잊으면 적어 둔 내용과 실제 서버가 어긋나고요.

오른쪽처럼 연결할 때 서버에 물으면, 서버가 지금 내주는 도구를 그대로 받아요. 이름과 설명과 입력 스키마가 목록에 함께 오니까, 도구 정보를 호스트 코드에 일일이 옮겨 적는 수고를 줄일 수 있습니다.

실제 예가 있어요. 판단 엔진 서버에는 나중에 구매 판단을 위한 도구 셋이 더해졌습니다. 그래도 에이전트 쪽 설정은 서버 주소 그대로예요. 연결할 때마다 새 목록을 받아 오니까요. 포털의 도구 시험 화면도 같은 생각을 따릅니다. 도구를 부르기 직전에 서버에서 목록을 다시 받아서, 서버가 지금 내주는 그대로를 기준으로 삼아요. 목록이 바뀌면 새 기능이 생긴 것을 알아볼 수 있죠. 다만 이름이 보인다고 바로 허용되는 것은 아닙니다. 지원하지 않는 기능이 있거나 권한 범위가 바뀌면 설정과 허용 목록도 검토해야 해요.

[SLIDE 92] 도구 정의 한 건: part_quotes

그 목록에 든 항목 하나를 그대로 옮겨 왔습니다. 화면 왼쪽이에요. 구매 쪽 업무 DB 서버가 내주는 부품 견적 조회 도구입니다.

맨 위가 도구의 이름이에요. 호출할 때 이 이름을 씁니다.

그 아래가 설명이고요. 부품별로 공급사의 단가, 불량률, 납기, 승인 여부를 읽는다고 적혀 있죠.

가운데 입력 스키마는 넘길 값을 정합니다. 부품 번호 하나를 글자로 받고, 생략하면 이 서버 구현에서는 펌프 축 씰 키트를 기본으로 씁니다. 값의 이름과 형, 그리고 비웠을 때 쓸 값까지 적혀 있으니, 모델이 무엇을 어떤 모양으로 넣을지 판단할 단서가 됩니다. 이 정의는 JSON Schema라는 표준 형식으로 적습니다. 스키마의 기본값 표시는 안내이고, 검사기가 빠진 값을 자동으로 채운다는 뜻은 아니에요. 실제 기본값 처리는 이 도구의 구현에서 합니다.

맨 아래에는 이 도구가 읽기만 한다는 표시가 붙어 있어요.

오른쪽에서 강조한 게 설명이죠. 모델이 이 도구를 부를지 말지 정할 때 읽는 게 바로 이 설명과 입력 스키마거든요. 그러니까 도구의 설명은 사람을 위한 주석에 그치지 않습니다. 설명이 모호하

면 모델이 엉뚱한 도구를 고르거나, 필요한 도구를 그냥 지나쳐요. 스킬의 설명이 스킬을 고르는기준이었던 것처럼, 도구의 설명은 도구를 고르는 기준입니다.

[SLIDE 93] 호출 한 건의 요청: 도구의 전체 이름과 입력

도구를 실제로 부를 때는 어떤 요청이 갈까요? 둘째 쿨러 경보의 실행 기록에서, 원인 진단을 부른 요청을 원래 모양 그대로 꺼냈어요.

첫 줄이 도구의 전체 이름입니다. 앞에 MCP 표시, 가운데에 서버 이름, 끝에 도구 이름이 있고, 이 실행 도구에서는 밑줄 두 개로 이어 표시했어요. MCP가 모든 제품에 이 이름 모양을 요구하는 것은 아닙니다. 판단 엔진 서버의 원인 진단 도구라는 뜻이죠.

그 아래가 입력입니다. 원인 진단 도구도 앞의 부품 견적 도구처럼 입력 스키마를 갖고 있고, 거기 정해 둔 항목이 설비, 경보의 종류, 경보 번호 셋이에요. 그 세 항목을 이름과 형에 맞춰 채운 JSON입니다.

앞에서 스킬의 한 줄에 이름이 셋 들어 있었죠. 서버 이름, 도구 이름, 넘길 값. 실제 요청에서는 앞의 두 이름이 전체 이름 하나로 합쳐지고, 넘길 값은 입력 JSON이 된 겁니다.

전체 이름에 서버 이름을 붙이는 데는 까닭이 있습니다. 같은 이름의 도구가 두 서버에 있을 수 있거든요. 앞에서 본 부품 견적 도구가 그래요. 업무 DB 전체 서버에도 있고, 구매 쪽 서버에도 있습니다. 앞에 서버 이름이 붙어 있으니 어느 서버의 것인지 헷갈리지 않죠.

그리고 입력 스키마는 넘길 값의 모양을 정한 계약이기도 합니다. 포털에서 도구를 시험해 볼 때, 입력이 스키마를 어기면 서버에 보내기도 전에 거절해요. 설비 칸에 숫자를 넣었거나 필요한 항목을 빠뜨린 요청은 서버까지 가지 않습니다. 형식이 맞지 않는 요청을 먼저 걸러 주는 겁니다. 하지만 문자열이라는 형식이 맞아도 다른 설비의 번호를 넣을 수 있죠. 스키마 검사를 통과한 것과 업무 대상을 정확히 고른 것은 따로 확인해야 합니다.

[SLIDE 94] 호출 한 건의 응답: mes_orders

이번에는 생산 오더 조회에서 돌아온 내용입니다. 화면은 HYD의 업무 DB 도구가 만든 업무 결과 본문이에요. 앞서 본 호출 이름의 모양처럼, 이 항목 이름도 HYD 사례와 MCP 공통 규약을 구분해서 읽어야 합니다.

맨 위 성공 표시는 이 업무 조회를 처리했다는 뜻입니다. 그 아래에는 어느 업무 시스템에서 가져왔는지와 기준 시각이 있어요. 판단용 값에는 오더 번호, 납기까지 남은 시간, 남은 수량이 들어 있고, 원래 행에는 그 값을 다시 확인할 자료가 담깁니다. 모든 서버가 이 항목 이름을 쓰는 것은 아닙니다.

바깥 MCP 응답도 함께 생각해 볼게요. 요청 자체를 처리하지 못한 프로토콜 오류가 있고, 요청은 전달됐지만 도구 실행에 실패해 오류 표시가 담긴 결과가 올 수도 있습니다. 그러니 결과가 돌아왔다는 사실만으로 업무가 성공했다고 판단하면 안 돼요. 포털 구현도 연결·프로토콜 상태와 도구 오류 표시를 따로 다룹니다.

업무 결과를 얻은 뒤에도 확인은 남습니다. 어느 설비의 어떤 조건을 조회했는지, 기록의 기준 시각이 판단에 충분히 가까운지 봐야 하죠. 성공 표시는 이번 요청의 처리 상태를 알려 줍니다. 답이 실제 업무 판단에 맞는지는 대상과 시각과 근거를 함께 대조해서 정합니다.

[SLIDE 95] 도구 호출 한 번이 지나는 길

요청과 응답을 봤으니, 호출 한 번이 처음부터 끝까지 지나는 길을 이어 보겠습니다. 화면에 칸이 다섯 개 있죠. 칸 사이에 적힌 작은 글자는 다음 칸으로 넘어갈 때 하는 일이에요. 요청, 확인, tools/call, 조회. 왼쪽부터 갈게요.

첫 칸, 모델이 하는 일은 어느 도구를 어떤 값으로 부르겠다는 요청을 내는 데까지입니다.

둘째 칸의 호스트가 그 요청을 받아서, 이 에이전트가 부를 수 있도록 허용된 도구인지 확인합니다. 허용된 요청이면 셋째 칸의 클라이언트가 tools/call로 서버에 전달하고요.

넷째 칸의 서버는 도구를 실행해서 마지막 칸의 시스템에서 값을 읽고, 응답을 만듭니다. 응답은 같은 길을 거슬러서 모델에게 돌아와요.

앞부분에서 에이전트는 데이터베이스에 직접 접속하지 않는다고 했죠. 그 말이 이 길에서는 이렇게 보입니다. 모델은 맨 왼쪽 칸에, 데이터베이스는 맨 오른쪽 칸에 있고, 이 HYD 도구 호출 경로에서는 그 사이의 세 역할을 거칩니다. 어떤 프로그램이든 반드시 이 구조만 쓴다는 뜻은 아니에요.

이 길에서 강조된 칸이 호스트입니다. 모델과 실행 사이에 호스트가 서 있죠. 그래서 모델이 무엇을 요청하든, 실제로 실행될 도구를 호스트에서 가를 수 있어요. 바로 이 자리에, 부를 수 있는 도구를 정해 둔 허용 목록이 놓입니다.

[SLIDE 96] 연결 방식 두 가지: stdio와 Streamable HTTP

메시지가 오가는 모양은 봤으니, 남은 건 클라이언트와 서버를 잇는 길입니다. 여기서는 표준 전송 방식 두 가지를 보겠습니다. 화면 왼쪽이 stdio, 오른쪽이 Streamable HTTP예요.

stdio는 표준 입력과 표준 출력을 뜻합니다. 호스트가 서버 프로그램을 자기 아래 하위 프로세스로 직접 띄워요. 그리고 그 프로세스의 표준 입력으로 요청을 넣고, 표준 출력으로 응답을 받습니다. 왼쪽 아래 적힌 것처럼, 서버가 이를 띄운 클라이언트와 같은 실행 환경에서 돌아요. 그 서버가 접속하는 DB까지 같은 PC에 있어야 한다는 뜻은 아닙니다.

Streamable HTTP는 서버가 먼저 떠 있는 방식입니다. 클라이언트가 그 서버의 주소로 접속해서, HTTP 요청으로 메시지를 주고받아요. 그래서 오른쪽 그림처럼 여러 클라이언트가 한 서버를 함께 쓸 수 있습니다.

어느 쪽을 고를지는 서버를 어디에 두고 누가 쓰느냐에 달려 있어요. 내 PC에서 나만 쓰는 도구라면 stdio가 간단하고요, 여러 에이전트가 함께 쓰는 공용 서버에는 HTTP 방식을 사용할 수 있습니다. HTTP 서버를 같은 PC에서 실행하는 구성도 가능해요. 어느 방식이든 실제 데이터 접근 권한은

별도로 정합니다. HYD는 두 방식을 다 써요. 다섯 서버 가운데 지식 그래프 서버 하나만 stdio이고, 나머지 넷은 미리 떠 있는 HTTP 서버입니다.

[SLIDE 97] 서버 설정 두 건: 명령으로 띄우는 서버와 주소로 접속하는 서버

두 방식은 설정만 봐도 바로 구별됩니다. 화면 왼쪽에 HYD의 실제 서버 설정 두 건을 옮겼어요.

위쪽 지식 그래프 서버는 실행할 명령과 인자로 적혀 있습니다. 호스트가 이 명령으로 서버를 직접 띄우는 거죠. stdio 방식이에요.

아래쪽 판단 엔진 서버는 주소로 적혀 있어요. 이미 떠 있는 서버에 접속하는 Streamable HTTP 방식입니다. 실제 설정에는 지식 그래프 서버의 접속 계정과 비밀번호 칸도 있는데, 화면에서는 뺐어요. 그런 비밀값을 설정에서 어떻게 다루는지는 안전 기준에서 따로 봅니다.

앞에서 stdio 서버는 호스트와 같은 PC에서 돈다고 했죠. 그게 실제로 문제가 된 기록이 있어요. 이 교재가 참고한 기록에서는 포털 백엔드 쪽에서 지식 그래프 서버를 시험해 보려다가, 띄울 명령을 찾지 못해서 실패했습니다. 이 서버는 에이전트 프로그램이 도는 PC에서 띄워지게 되어 있었거든요. 그러니까 stdio 서버는 그걸 띄우는 쪽의 실행 환경에서 명령과 패키지를 찾을 수 있어야 합니다.

이렇게 설정 한 줄이 연결의 길과 서버가 도는 자리를 정합니다. 그 길로 연결한 호스트는 목록을 받아서 도구를 알고, 입력 스키마에 맞춘 값을 넘기고, 정해진 모양의 응답을 받아요.

[SLIDE 98] 스킬과 MCP: 수행 방법과 실행 도구

연결의 길까지 따라왔으니, 열두 번째 단원에서는 지금까지 따로따로 본 스킬과 MCP를, 한 실행 안에 함께 놓아 보겠습니다. 스킬이 있는데 왜 서버를 따로 연결해야 할까요? 거꾸로, 서버가 있는데 왜 스킬이 필요할까요? 한 실행 안에서 둘의 몫을 나눠 보면 답이 보입니다.

[SLIDE 99] 스킬과 MCP의 역할 구분

둘을 나란히 놓았습니다. 왼쪽 스킬은 수행 방법이고, 오른쪽 MCP는 실행 도구를 잇는 연결이에요.

왼쪽 스킬은 수행 방법을 적은 지침입니다. 사람이 읽고 고칠 수 있는 글이고, 지침 문장 자체가 호출을 실행하지는 않아요. 스킬 묶음에 실행 스크립트가 있어도 그것을 읽고 호출할 실행 주체와 권한이 따로 필요합니다. 원인 진단 도구로 원인 1순위를 얻으라고 적어 둘 뿐이죠.

오른쪽 MCP는 도구를 내주고 호출을 받는 연결입니다. 실제로 일을 하는 건 서버라는 프로그램이에요. 그런데 어떤 순서로 부를지는 정하지 않습니다. 판단 엔진 서버는 원인 진단을 부르면 실행해서 응답할 뿐이에요.

한 문장으로 하면 이렇습니다. 스킬은 무엇을 어떤 순서로 할지를 적고, MCP는 그 글에 적힌 도구를 실제로 부를 수 있게 이어 준다. 이렇게 방법과 실행이 나뉘어 있으니까, 방법을 바꿀 때는 지침

을, 접속 위치를 바꿀 때는 연결 설정을 먼저 살펴볼 수 있습니다. 도구의 이름이나 입력 형식까지 바뀌면 지침과 연결을 함께 맞춰야 하고요.

[SLIDE 100] 둘 가운데 하나만 있을 때

이 냉각 대응 업무에서 지침과 도구 중 하나가 빠지면 어떻게 될까요? 첫 카드는 필요한 도구를 연결하지 않은 경우입니다. 지침에 원인을 진단하라고 적혀 있어도 현재 설비 값을 가져올 길이 없으면 이 업무를 끝낼 수 없습니다. 반대로 글만으로 처리할 수 있는 일까지 언제나 MCP가 필요한 것은 아니에요.

둘째 카드는 도구는 있지만 업무 방법을 주지 않은 경우입니다. 호출은 가능해도 어떤 순서로 확인하고 무엇을 제출할지 모델이 그때그때 정할 부분이 커집니다. 방법은 스킬 외에도 프롬프트나 고정된 프로그램으로 줄 수 있어요. 여기서는 같은 방법을 재사용하기 위해 스킬로 관리하는 겁니다.

셋째 카드는 지침과 필요한 도구를 함께 갖춘 경우예요. 어떤 일을 할지와 실제 정보를 얻을 길이 마련됩니다. 그 상태에서도 권한, 입력, 응답을 확인해야 하고, 지침대로 실행됐는지는 기록으로 검토합니다. 둘을 연결한 것만으로 정답이 보장되지는 않아요.

예를 들어 지침에서 생산 오더 조회 도구를 지정했는데 그 도구가 있는 업무 DB 서버를 연결하지 않았다면, 그 호출 경로부터 보완해야 합니다. 반대로 연결과 입력이 정상인데 엉뚱한 업무 순서를 따랐다면 지침을 살펴보겠죠. 문제가 생긴 자리를 가려 고칠 수 있다는 것이 역할을 나눈 실용적인 가치입니다.

[SLIDE 101] 한 실행에서 본 스킬의 단계와 서버의 응답

둘째 쿨러 경보의 실행 기록을 스킬 단계와 대응시켜 놓았습니다. 이 표의 줄은 절차 번호 순서이며 실제 호출의 시간순은 아닙니다. 실제로는 원인 진단 뒤 후보 비교를 먼저 부르고, 그다음 업무 값을 확인했습니다. 표의 왼쪽 칸이 스킬, 가운데와 오른쪽 칸이 서버의 몫이에요.

첫 줄, 2단계 원인 진단은 판단 엔진 서버가 실행해서, 원인 1순위로 쿨러 핀 오염을 돌려줬습니다. 보류 표시가 없었으니, 보류하지 않고 다음 도구로 넘어갔죠.

둘째 줄, 3단계 사실 확인은 업무 DB 서버가 맡았어요. 납기 오더와 출하 대기 로트를 돌려줬죠.

셋째 줄, 4단계 후보 비교는 다시 판단 엔진 서버의 몫이었는데요, 응답이 너무 커서 본문이 에이전트에게 전달되지 않았습니다.

그럼 후보 비교의 결과를 못 받고 어떻게 판단을 냈을까요? 넷째 줄을 보세요. 이 실행은 절차에 없던 조치 효과 예측을 세 번 따로 불러서, 조치 안마다 유온이 어떻게 될지를 받았어요. 그리고 마지막 줄, 5단계의 판단 제출 도구는 서버 안에서 후보 비교를 다시 해서 순위를 매긴 묶음을 제출하고, 판단 번호와 함께 추천과 설명을 돌려줬습니다. 에이전트는 이 설명과 예측 값을 근거로 지는 안의 까닭을 적었고요.

이 표를 위에서 아래로 읽으면 스킬의 방법을 따라가는 거고, 왼쪽에서 오른쪽으로 읽으면 그 방법이 어느 서버의 실행으로 바뀌었는지가 보입니다. 절차 밖의 넷째 줄도 마찬가지예요. 무엇을 더 부를지는 에이전트가 정했지만, 값을 돌려준 건 역시 서버였습니다.

이어서 이렇게 연결한 도구를 에이전트에게 어디까지 허락할지, 그리고 4단계처럼 응답이 다 오지 못했을 때 무엇을 남겨야 하는지, 연결의 안전 기준을 다룹니다.

[SLIDE 102] 도구 권한과 연결 검사

이번 시간에는 여러 도구를 연결할 때 지킬 안전 기준을 다룹니다. 앞에서 한 실행 안에서 스킬과 MCP가 각자 무엇을 맡는지 나눠 봤죠. 이번에는 연결한 도구의 권한부터, 파일과 비밀값과 불완전한 응답을 어떻게 다루는지까지 봅니다. 이어서 마지막 도구 연결 실습에서 무엇을 확인하고 남길지 함께 안내합니다. 실제 실습을 마친 뒤에는 복습과 퀴즈로 돌아옵니다.

열세 번째 단원부터 시작할게요. MCP로 서버를 하나 더 연결하면, 에이전트가 할 수 있는 일이 늘어나죠. 그런데 잘못했을 때 생기는 일도 같이 늘어납니다. 연결한 도구 가운데 어디까지 에이전트에게 허락해야 할까요? 읽는 도구와 쓰는 도구의 차이부터 보고, 에이전트가 부를 수 있는 도구를 정하는 장치까지 보겠습니다.

[SLIDE 103] 읽는 도구와 쓰는 도구: 잘못 불렀을 때의 차이

도구를 연결할 때는 어떤 데이터를 읽는지와 무엇을 바꾸는지를 먼저 봅니다. 화면 왼쪽은 읽는 도구, 오른쪽은 쓰는 도구예요.

생산 오더 조회는 업무 기록을 바꾸지 않고 값을 가져옵니다. 그렇다고 잘못 불러도 아무 영향이 없는 것은 아니에요. 읽을 필요가 없는 다른 부서의 계약이나 개인정보를 가져오면 정보가 노출될 수 있습니다. 또 잘못된 설비의 값을 받아 판단에 쓰면 이후의 추천도 틀어집니다. 읽기에서도 대상과 범위를 제한해야 하는 이유입니다.

메일 발송처럼 쓰는 도구는 시스템을 바꾸거나 외부에 결과를 남깁니다. 이미 보낸 메일은 요청을 취소한다고 상대의 받은편지함에서 사라지지 않죠. 어떤 변경인지, 누가 허락하는지, 중복 호출을 어떻게 다룰지까지 확인해야 합니다.

이 구분은 도구가 미치는 영향을 살펴보는 출발점입니다. 읽기인지 쓰기인지 표시를 확인하고, 실제 구현과 접근 계정이 그 범위에 맞는지도 봐야 해요. 다음 장에서는 서버가 이 성격을 어떤 표시로 알려 주는지 보겠습니다.

[SLIDE 104] readOnlyHint: 서버가 도구에 다는 읽기 전용 표시

그럼 어떤 도구가 읽는 도구인지는 어떻게 알까요? MCP에는 도구 정의에 붙이는 표시가 있습니다. 화면 첫 줄의 readOnlyHint, 읽기 전용 표시예요.

이 표시는 도구 정의의 주석 칸에 붙고, true면 시스템을 바꾸지 않는 도구라는 뜻입니다. 앞에서 본 부품 견적 도구에도 붙어 있었죠.

둘째 줄을 보세요. 판단 엔진 서버는 판단을 제출하는 두 도구만 빼고 모두 true예요. 그 두 도구가 false인데, 하나는 조치 추천안을, 하나는 구매 추천안을 제출하죠. 셋째 줄, 업무 DB 서버의 도구는 모두 true고요.

그런데 넷째 줄이 중요합니다. 이 표시는 이름 그대로 힌트예요. 서버가 스스로 적은 값이라서, 서버를 만든 사람이 잘못 적을 수도 있고 일부러 속일 수도 있습니다. 그래서 HYD 포털은 표시와 함께 도구 이름까지 한 번 더 검사합니다. 표시는 true인데 이름이 쓰기나 삭제를 뜻하면, 읽는 도구로 자동 인정하지 않습니다. 다만 이름 검사도 잘못된 구현이나 거짓 표시를 모두 밝혀 내는 증명은 아니에요. 판단 엔진 서버의 판단 제출 도구는 아예 표시부터 false죠. 그래서 포털의 시험 화면에서는 부를 수 없고, 처리 건 안의 에이전트만 부릅니다.

읽기 도구 쪽도 표시에만 기대지 않습니다. 넷째 줄 끝에 적은 것처럼, 업무 DB와 시계열 DB를 읽는 서버는 뒤에서 읽기 전용으로 접속해요. 그러니까 표시는 도구를 고르는 기준이고, 실제로 읽기만 하게 막는 일은 서버 뒤의 장치가 함께 맡는 거예요.

[SLIDE 105] 등록한 서버의 도구가 지나는 세 겹

이 표시가 실제로 어디서 쓰이는지는 실습에서 직접 겪게 됩니다. HYD 포털에 서버를 직접 등록하면, 그 서버의 도구는 세 겹을 지나야 에이전트에 닿아요. 그림 위쪽에 번호가 붙은 세 띠가 그 세 겹입니다.

첫째 겹은 서버가 도구에 다는 표시예요. 왼쪽 띠에 도구가 셋 있죠. 읽기로 표시한 도구, 쓰기로 표시한 도구, 아무 표시가 없는 도구입니다.

둘째 겹은 연결 검사예요. 포털이 서버에 실제로 연결해서 도구 목록을 받아 보고, 읽기 표시가 있으면서 이름도 쓰기를 뜻하지 않는 도구만 통과시킵니다. 쓰기로 표시한 도구는 여기서 걸러져요. 표시가 아예 없는 도구도 걸러집니다. 읽는 도구인지 알 수 없으니까요. 다만 HYD에는 표시만 빠진 도구를 강사가 읽기로 확인하는 별도 절차가 있습니다. 쓰기 표시가 있거나 이름이 쓰기를 뜻하는 도구에는 이 예외를 적용하지 않습니다. 그림은 별도 확인이 없는 기본 판정입니다.

셋째 겹이 허용 목록입니다. 검사를 통과한 도구의 이름만 이 목록에 오르고, 오른쪽의 에이전트는 목록에 있는 도구만 불러요. 그림에서 세 도구를 왼쪽부터 따라가 보면, 끝까지 이어지는 건 맨 위의 부품 견적 도구 하나뿐이죠. 나머지 둘은 가운데 띠에서 멈춥니다.

그리고 연결 검사는 한 번 하고 끝나는 게 아닙니다. 서버 설정을 고치면 검사 통과 기록이 사라지고, 다시 검사해야 에이전트에 붙어요. 검사한 설정과 지금 설정이 같을 때만 통과를 인정하는 거죠. 이 검사가 서버 내부의 모든 동작을 시험하거나 이후 변경까지 보증하는 것은 아닙니다. 목록 확인, 호출 시험, 실제 업무 결과 검증의 범위를 나눠 봐야 합니다.

[SLIDE 106] 연결 검사의 실패 사유와 분류

그럼 둘째 겹인 연결 검사를 통과하지 못하면 어떻게 될까요? 실습에서 서버를 등록하다 보면 연결 검사가 실패할 수 있는데, 그럴 때 무엇을 보면 되는지 표로 정리했습니다.

먼저 원칙이 하나 있어요. 연결 검사는 도구 목록을 받지 못했을 때 빈 목록을 성공처럼 돌려주지 않습니다. 실패로 알리고, 무엇이 막혔는지를 분류와 사람이 읽는 사유로 함께 줘요. 표의 왼쪽 칸이 그 분류입니다.

위의 두 줄은 서버를 찾는 단계의 실패예요. 첫째는 접속을 거절당한 경우입니다. 서버가 켜져 있는지, 포트가 맞는지 봐요. 둘째는 이름을 해석하지 못하거나 네트워크 경로에 닿지 못한 경우고요. 서비스 이름이나 호스트 이름을 확인합니다.

셋째 줄이 시간 초과예요. 정해진 시간 안에 응답이 없으면 실패입니다. 포털은 목록을 받을 때도, 도구를 부를 때도 정해진 몇 초까지만 기다리고, 부를 때를 조금 더 길게 잡아 뒀어요. 그 안에 답이 없다면 서버가 느리거나, 방화벽이 막고 있을 수 있습니다.

넷째는 인증 실패예요. 자격 증명이 없거나 만료됐는지, 그 자격에 필요한 접근 권한이 있는지 살핍니다. 사유를 확인하고 자격 증명과 권한을 바로잡은 뒤 다시 검사해요. 다섯째는 주소의 경로가 MCP 끝점이 아닌 경우고요, 여섯째는 stdio 서버의 명령을 찾지 못한 경우예요. 여섯째는 앞에서 본 것처럼, stdio 서버는 그걸 띄우는 쪽 PC에서 실행된다는 점을 떠올리면 됩니다.

이렇게 분류가 있으면 고칠 곳이 좁혀져요. 그냥 「연결 실패」라고만 나오면 서버, 주소, 토큰, 경로를 전부 의심해야 하잖아요. 사유가 다음에 확인할 곳을 가리켜 주니까, 실습 중에 막혀도 어디부터 볼지 알 수 있습니다.

[SLIDE 107] 허용 목록(Allowlist): 에이전트가 부를 수 있는 도구의 이름

셋째 겹인 허용 목록이 하는 일은 멈춤 조건에서 이미 봤죠. 사전 허용한 범위를 넘는 도구 요청은 호스트의 권한 검사에서 다룬다고 했습니다. 구체적으로 추가 확인을 요구하는지, 명시적으로 막는지는 실행 정책에 따라 달라요. 화면 왼쪽이 그 목록의 실제 내용이에요.

맨 위 두 줄은 지식 그래프 서버의 조회 도구 둘입니다. 이 서버는 도구 이름을 하나씩 적어서 조회만 허용했어요. 사실 이 서버에는 그래프를 고치는 도구도 있습니다. 그런데 서버 설정에서 읽기 전용으로 띄워서, 그 도구까지 막아 뒀어요.

그 아래 네 줄 끝에 별표가 붙어 있죠? 별표는 그 서버가 내주는 도구 전부라는 뜻입니다. 판단 엔진 서버와 업무 DB 서버들처럼, 미리 검사해 둔 기준 서버에만 별표를 써요. 그래서 판단 엔진 서버의 별표 안에는, 앞에서 본 판단 제출 도구 둘도 들어 있습니다.

맨 아래 줄은 파일 읽기, 스킬 읽기 같은 호스트 자신의 도구입니다.

오른쪽에서 강조한 게 학생이 등록한 서버예요. 학생이 새로 등록한 서버에는 별표를 쓰지 않습니다. 연결 검사에서 읽기로 인정된 도구의 이름만 하나씩 오르고, 읽기로 인정되지 않은 도구는 실행을 막습니다. 모델이 그런 호출을 요청하는 것과 실제로 실행 권한이 주어지는 것은 별개예요. 결국 이 목록이 곧 에이전트가 쓰는 도구의 범위예요.

[SLIDE 108] 쓰기의 세 층: 판단의 제출, 설비 명령, 업무 기록과 발송

별표 안에 쓰기 도구가 들어 있다면, 에이전트는 어디까지 쓸 수 있는 걸까요? 쓰기에도 층이 있습니다. 화면의 카드 세 장이 그 세 층이에요.

첫 카드는 판단의 제출입니다. 판단 엔진 서버의 쓰기 도구 둘이 여기 해당해요. 이 도구들이 쓰는 건 사람이 고를 추천안입니다. 설비 명령이나 발주·작업지시를 실행하는 쓰기와는 구별됩니다. 이 HYD 사례에서 에이전트에 연결된 MCP 쓰기 도구는 판단 제출용 두 개이고, 작업 폴더의 결과 파일도 따로 작성합니다.

두 번째 카드는 설비 명령이에요. 팬 속도를 바꾸는 명령은 담당자가 승인한 뒤에야 만들어집니다.

세 번째 카드는 업무 기록과 메일입니다. 작업지시를 등록하고 메일을 보내는 일이죠. 이것도 승인 뒤의 시스템이 해요. 메일 발송 도구는 아예 에이전트에 연결하지 않습니다.

추천은 에이전트가, 승인은 사람이, 실행은 시스템이 한다는 경계가 있었죠. 이 경계가 지침의 문장에만 있는 게 아닙니다. 어느 도구를 누구에게 연결했는지에도 놓여 있어요. 그래서 스킬에 실수로 메일을 보내라고 적혀 있더라도, 이 MCP 경로에는 메일을 실행할 도구가 없습니다. 같은 효과를 내는 다른 도구나 직접 접속 경로까지 없는지는 전체 실행 권한에서 확인해야 합니다.

[SLIDE 109] 최소 권한 원칙: Principle of Least Privilege

읽기에서도 대상과 범위를 제한해야 한다고 했죠. 이를 일관되게 적용하는 기준이 무엇일까요? 보안에는 이 물음에 답하는 원칙이 있습니다. 최소 권한 원칙이에요.

첫 줄의 정의를 보세요. 최소 권한 원칙은 맡은 일에 필요한 권한만 주고, 그 밖의 권한은 주지 않는다는 원칙입니다. 읽기 도구라도 그 일에 안 쓰이면 연결하지 않아요.

HYD의 업무 DB 서버가 셋으로 나뉜 까닭이 여기 있습니다. 둘째 줄, 정기 정비 계획 에이전트에는 정비 쪽 서버만 연결해서 정비 도구 네 개만 받게 했어요. 셋째 줄, 예비품 구매 에이전트에는 구매 쪽 서버만 연결해서 구매 도구 세 개만 받게 했고요.

두 서버의 도구는 서로 겹치지 않습니다. 그리고 나눈 서버에는 SQL을 자유롭게 쓰는 도구를 두지 않았어요. 그런 도구가 있으면 정해 둔 범위를 넘어 다른 표까지 읽을 수 있거든요. 읽기라도 범위를 좁혀 두면, 에이전트가 잘못 판단했을 때 닿을 수 있는 곳도 좁아집니다.

[SLIDE 110] 파일 범위 · 비밀값 · 불완전한 응답

이제 마지막 이론 단원입니다. 허용 목록과 연결 검사로 에이전트가 부를 도구는 정했어요. 그런데 도구 권한 밖에서 새는 곳은 없을까요? 그리고 도구가 답을 다 주지 못했을 때는 무엇을 남겨야 할까요? 에이전트가 읽고 쓰는 파일의 범위, 서버에 연결할 때 쓰는 비밀값, 불완전한 응답의 처리를 차례로 짚어 보겠습니다.

[SLIDE 111] 파일 접근 범위: 작업 폴더 하나

에이전트는 파일을 읽고 쓰는 도구도 씁니다. 스킬 파일을 읽고, 결과 파일을 쓰죠. 그럼 그 범위는 어디까지일까요?

첫 카드가 범위예요. 에이전트는 실행마다 작업 폴더 하나를 받고, 그 폴더 밖의 파일은 읽거나 고치지 말라는 지시를 받습니다. 카드에 따옴표로 적힌 문장이 그 지시의 원문이에요.

두 번째 카드는 금지입니다. 홈 디렉터리를 찾아보지 말라는 지시가 따로 있어요. 홈에는 프로그램의 설정과 로그인 정보가 들어 있거든요.

세 번째 카드는 결과의 자리예요. 결과를 쓰는 파일은 정해진 경로 하나뿐입니다.

여기서 정확히 말해 둘 게 있어요. 작업 폴더를 나누고 지시하는 것 자체는 운영체제의 강제 격리가 아닙니다. 실제 접근을 막으려면 실행 계정 권한, 허용 경로 검사, 격리 환경 같은 별도의 통제가 필요해요. 지금 화면은 HYD의 지침과 결과 저장 위치를 설명한 것입니다. 그래서 범위를 정하는 것과 함께, 에이전트가 닿을 수 있는 곳에 아예 비밀을 두지 않는 일이 필요해요.

[SLIDE 112] 인증 비밀값: 설정에는 자리표시자, 값은 실행 직전에

그런 비밀 가운데 하나가, 서버에 연결할 때 쓰는 인증 비밀값입니다. 외부 서비스에 연결하려면 토큰 같은 값이 필요하죠. 그런데 이 값을 서버 설정에 그대로 적으면, 설정을 보는 사람 누구에게나 드러나요.

그래서 설정에는 값 대신 자리표시자를 둡니다. 왼쪽 위 설정에서 인증 머리말을 보세요. 토큰이 들어갈 자리에 값이 아니라, 비밀값이라는 표시와 그 값의 이름만 적혀 있죠. 실제 값은 설정과 따로 보관했다가, 실행 직전에만 채웁니다.

값이 준비되지 않았으면 어떻게 할까요? 빈 토큰으로 연결을 시도하지 않아요. 그 서버를 그 실행에서 뺍니다. 그리고 화면이나 기록에 접속 정보가 보일 때는, 왼쪽 아래처럼 값을 별표로 가려요. 연결 방식을 볼 때 지식 그래프 서버의 설정에서 뺐던 비밀번호 칸이, 포털 화면에는 이렇게 가려진 채로 보입니다.

자리표시자를 쓰면 토큰을 바꿀 때도 편합니다. 토큰이 만료돼서 새 값으로 바꿔도, 설정에 적힌 건 자리표시자 그대로잖아요. HYD는 자리표시자가 든 설정으로 검사 지문을 계산하므로, 토큰 값만 바꿔도 기존 검사 표시는 유지될 수 있습니다. 하지만 새 토큰의 계정과 권한까지 예전과 같다는 뜻은 아니에요. 실제 접속과 필요한 권한은 확인해야 합니다.

에이전트 쪽에도 지킬 비밀이 있어요. 오른쪽 넷째 줄입니다. 에이전트 프로그램을 띄우는 쪽은 데이터베이스 접속 문자열과 비밀번호를 갖고 있어요. 그중 워커만 쓸 DB 비밀번호 같은 값은 자식 프로그램에 상속되지 않도록 환경변수에서 지웁니다. 모델 서비스 로그인처럼 실행에 필요한 값은 남는 예외가 있어요. 에이전트가 그 값을 읽으면, 허용 목록을 거치지 않고 데이터베이스에 바로 접속할 수 있거든요. 또 MCP 연결에 필요한 비밀값은 실행 중 설정에 들어갈 수 있고, 종료 뒤 정리하는 코드가 있습니다. 자리표시자와 화면 가림은 노출을 줄이는 방법이지, 실행 중 모든 비밀이

에이전트에게 보이지 않는다는 보장은 아닙니다. 무엇이 어느 과정에 전달되는지까지 확인해야합니다.

[SLIDE 113] 외부 응답은 불완전할 수 있다: 잘린 결과의 실제 사례

도구의 응답을 온전히 읽지 못한 경우에는 무엇을 남겨야 할까요? 표에는 두 쿨러 경보의 기록을 나란히 정리했습니다.

첫 줄이 공통 상황입니다. 두 실행 모두 조치 후보 비교 응답이 12만 자를 넘었고, 실행 환경이 본문을 다른 파일로 보관했습니다. 에이전트가 원래 호출 응답에서 상세 내용을 직접 받지 못한 거예요. 이것은 이 실행 환경의 전달 제한이지, MCP가 모든 응답에 같은 글자 수 한도를 정한다는 뜻은 아닙니다.

둘째 줄의 첫 실행에서는 필요한 항목만 좁혀 다시 뽑겠다는 말이 남았습니다. 이것은 앞으로 할 일에 대한 계획입니다. 그 말만 보고 실제로 다시 조회해서 성공했다고 기록하면 안 됩니다. 실행한 호출과 돌아온 결과가 있어야 그 다음을 말할 수 있어요.

셋째 줄의 다른 실행은 열지 못한 상세 출력이 있음을 마지막 보고에 밝혔습니다. 판단 제출 결과의 설명과 별도로 조회한 예측 값에 근거해 보고했다는 범위도 함께 남겼고요.

받은 결과, 다시 확인하겠다는 계획, 끝내 읽지 못한 범위를 나누는 이유는 보고를 읽는 사람이 무엇을 근거로 판단했는지 알게 하려는 것입니다. 설명이 그럴듯한지만 보지 말고, 그 설명을 뒷받침한 자료가 실제로 전달됐는지를 확인해야 합니다.

[SLIDE 114] 값 없음 · 실패 · 시간 초과 · 잘림의 구분

응답이 기대와 다를 때 결과에 어떻게 적어야 하는지, 표에 네 경우로 정리했습니다.

위의 두 줄은 멈춤 조건에서 이미 구분한 거예요. 정상 조회에서 0행이면 그 조건에 맞는 행을 찾지 못했다고 적습니다. 재고 수량이 0이라는 뜻은 아니에요. 호출 실패나 도구 오류가 확인됐다면 사유를 적고, 0이나 빈 값으로 바꾸지 않습니다.

도구를 연결해서 쓰면 경우가 둘 더 생깁니다. 강조된 셋째 줄이 정해진 시간 안에 응답하지 못한 경우예요. 연결 검사에서 본 시간 초과가, 도구를 부를 때도 생기는 거죠. 업무 DB 조회처럼 시간 제한이 걸린 호출은, 시간을 넘기면 실패로 돌아옵니다. 이때 확인되는 것은 제한 시간 안에 결과를 받지 못했다는 사실입니다. 서버가 작업을 전혀 하지 않았다고 단정할 수는 없어요. 특히 변경을 남기는 도구라면 재시도 전에 처리 상태와 중복 방지 여부부터 확인해야 합니다.

넷째 줄은 응답이 잘린 경우입니다. 본문의 일부만 오거나 아예 전달되지 않으면, 읽지 못한 범위를 밝혀요. 포털의 도구 시험 화면도 같은 원칙을 따릅니다. 결과가 정해진 길이를 넘으면 잘라서 보여 주되, 잘렸다는 표시와 원래 크기를 같이 알려 줘요.

네 경우 모두 지키는 원칙은 하나예요. 모르는 걸 아는 것처럼 적지 않는다. 시간을 넘긴 조회를 0으로 적으면 어떻게 될까요? 모르는 재고가 없는 재고로 바뀌고, 잘못된 발주로 이어질 수 있습니다.

[SLIDE 115] 근거 표시: 호출 기록과 값의 출처

마지막 기준은 근거 표시입니다. 첫째 쿨러 경보의 실행 기록 한 줄을, 이번에는 근거 표시라는 눈으로 다시 읽어 볼게요. 화면 왼쪽이 그 한 줄이에요.

기록에는 언제 불렀고 얼마나 걸렸는지, 어느 서버의 어느 도구를 불렀는지가 남아 있습니다. 판단 엔진 서버의 원인 진단 도구를 불렀고요, 넘긴 입력과 돌아온 원문이 나란히 있어요.

그런데 여기에 겹이 하나 더 있습니다. 오른쪽 맨 아래, 값의 출처예요. HYD의 판단 결과에는 값을 읽은 원천과 조회 근거가 함께 남습니다. 관측 시각과 조회 시각은 같은 뜻이 아니므로 각 항목이 나타내는 시각을 확인해야 해요. 앞서 본 생산 오더의 납기 값은 MES 자료에서 왔다는 것을 따로 추적할 수 있습니다. 왼쪽 원인 진단 한 줄만으로 그 납기 값까지 확인했다고 읽지는 마세요. 판단 엔진의 출처 정보에는 원천과 표, 조건, 실행한 조회문 등이 담깁니다. 모든 외부 MCP 서버가 이런 출처 항목을 자동으로 준다는 뜻은 아닙니다.

서버를 여러 개 연결할수록 이 두 겹의 표시가 중요해집니다. 어느 서버가 준 값인지, 그 서버는 어디서 그 값을 읽었는지가 남아 있어야, 사람이 추천을 승인하기 전에 근거를 되짚을 수 있거든요. 그러니까 도구를 안전하게 연결한다는 건 두 가지예요. 권한 밖으로 새는 곳을 막는 일, 그리고 받은 것과 받지 못한 것을 추적할 수 있게 남기는 일입니다.

[SLIDE 116~117] 실습: HYD 업무 도구의 연결과 호출

지금까지 스킬이 부르는 도구가 MCP로 어떻게 이어지는지, 그리고 그 연결에 어떤 안전 기준이 붙는지를 따라왔죠. 이제 그걸 직접 해 볼 차례입니다. 마지막 실습의 포털 단계에서는 HYD 포털에서 업무 도구를 직접 연결하고 불러 봅니다. 먼저 이미 등록된 업무 DB 서버와 판단 엔진 서버를 열어서, 연결 방식과 도구 목록을 읽어요. 그다음 등록된 업무 서버의 연결 검사를 확인하고, 허용된 읽기 도구 하나를 호출합니다. 호출한 도구는 기존 에이전트에 연결하고요.

실습이 끝나면 두 가지가 남습니다. 연결 검사를 통과한 기록, 그리고 도구 이름과 입력 값과 반환된 행이 적힌 호출 기록 한 줄이에요. 쓰기 도구가 왜 호출 대상에서 빠졌는지 앞에서 본 세 겹으로 설명할 수 있으면 성공입니다.

이것이 마지막 실습의 포털 단계에서 확인할 내용입니다. 이어서 같은 실습의 로컬 단계도 살펴본 뒤 실제 화면으로 넘어가겠습니다.

[SLIDE 118~119] 실습: 서로 다른 MCP 서버의 연결과 조합

같은 실습의 로컬 단계는 여러 서버를 조합하는 활동입니다. 앞 단계에서 확인할 포털의 연결 검사와 비교해 보면 차이가 드러납니다. 같은 실습의 로컬 단계에서는 자리를 내 PC의 개발 환경으로 옮겨서, 서로 다른 MCP 서버 여러 개를 연결하고 조합해 봅니다. 이번 실습은 바이브 코딩으로 진행해요. 미리 준비된 파일시스템, Git, 시간 서버를 각각 연결하고, 서버마다 실행 방법과 접근 권한을 정합니다.

연결한 다음에는 서버마다 도구 목록과 호출 결과가 어떻게 다른지 견줘 보겠습니다. 그리고 파일을 읽고, Git의 변경 기록을 확인하고, 시간 정보를 확인하는 일을 한 요청으로 이어서 불러요. 그 조합 결과가 이 실습에서 남는 산출물입니다.

성공 기준은 연결 방식의 차이를 설명하는 거예요. HYD 포털에는 방금 배운 등록·검사 절차가 있습니다. 로컬 코딩 도구에서는 해당 도구가 제공하는 서버 설정과 실행 권한을 확인해야 합니다. 이 차이를 설명할 수 있으면 됩니다.

서버별 결과가 한 요청 안에서 어떻게 사용됐는지도 호출 기록으로 남기겠습니다.

[SLIDE 120] 쿨러 사건의 94초를 다시 읽기

에이전트 구성과 원인 추론 실습에서 구성한 결과를 떠올리면서, 처음의 쿨러 사건을 다시 읽어 보겠습니다. 에이전트가 무엇을 확인했고 무엇을 사람에게 넘겼는지, 표의 다섯 장면으로 비교해 볼게요. 가운데 열의 물음에 먼저 답한 뒤 오른쪽 근거와 대조해 보세요.

첫 줄, 대안 평가가 네 후보의 점수를 냈어요. 계산한 건 누굴까요? 모델이 아니라 판단 도구입니다. 모델은 어느 도구에 뭘 넣을지 적었을 뿐이죠.

둘째 줄의 70 %와 약 46퍼센트, 둘 다 지금의 값일까요? 아니에요. 70은 지식 그래프에 적힌 기준이고, 약 46퍼센트가 그때 읽은 값입니다.

셋째 줄의 쿨러 핀 오염과 팬 구동 불량은 둘 다 원인 후보였어요. 이 모델의 증거 기준이 지지한 것은 쿨러 핀 오염이었습니다. 팬 진동이 기준보다 낮아서 팬 구동 불량은 지웠죠.

넷째 줄, 추천안 제출은 에이전트가 했고, 팬 속도 명령은 운전원이 승인한 뒤 시스템이 냈습니다. 마지막 줄의 조회 0행은 그 조건에 맞는 기록이 없다는 뜻이고, 조회 실패는 기록을 확인하지 못했다는 뜻이에요. 둘 다 재고 수량 0과는 구분해야 합니다.

계산의 주체, 지식과 값, 확인과 가설, 제안과 실행, 일치하는 기록 없음과 조회 실패. 이 다섯 짝을 가를 수 있으면, 에이전트가 낸 결과를 어디까지 믿어도 되는지 판단할 수 있습니다.

[SLIDE 121] 복습: 무엇을 어디에 둘 것인가

에이전트의 실행 흐름에 이어, 스킬과 도구 연결을 돌아보겠습니다. 실습에서 실제로 확인한 기록과 남은 미확인을 바탕으로, 이번 회차에 배운 걸 실제 상황에 적용해 보겠습니다. 표의 왼쪽 칸에, 에이전트에게 일을 맡기다 만나는 상황 네 가지를 모았어요. 각 상황에서 먼저 볼 자리를 스킬, SOP, 도구 목록, 자리표시자 가운데서 골라 보세요.

첫째, 정기 정비 경보도 매번 같은 방법으로 처리하게 하고 싶을 때. 둘째, 운전원이 팬을 100 %로 올리는 작업 순서를 찾을 때. 셋째, 판단 엔진 서버의 도구가 셋 늘었을 때. 넷째, 토큰이 필요한 외부 서버를 등록할 때입니다.

가운데 칸과 맞춰 볼게요. 첫째는 스킬입니다. 반복되는 업무의 방법은 이름 있는 지침으로 남기고, 그 일을 맡을 에이전트에 붙이죠. 프롬프트에 매번 적는 게 아니고요.

둘째는 SOP예요. 운전원이 읽는, 설비에 하는 일의 절차죠. 똑같이 「절차」라는 말이 붙어도, 에이전트 스킬에서 찾을 게 아닙니다.

셋째는 도구 목록입니다. 서버가 지금 내주는 기능을 목록에서 확인하고 필요한 권한과 입력 형식도 함께 살펴봅니다. 넷째는 자리표시자예요. 토큰 값은 설정 밖에 두고, 실행 직전에 채웁니다.

돌아보면 네 상황 모두 무엇을 어디에 둘지의 문제였어요. 방법은 스킬에, 사람의 작업 절차는 SOP에, 도구의 정보는 서버가 내주는 목록에, 비밀은 설정 밖에 둡니다. 자리를 맞게 구분하면, 바꿀 일이 생겼을 때 무엇부터 점검할지 알 수 있습니다.

[SLIDE 122] 오늘 다룬 핵심 개념과 주요 용어 정리

사건 쪽은 이렇게 다시 읽어 봤고요. 다음은 이번 회차에 나온 용어를 정리해 보겠습니다. 표에 이번 회차에 다룬 아홉 개념의 이름과 뜻이 있어요.

LLM은 입력과 학습한 내용을 바탕으로 응답을 만드는 대규모 언어 모델이에요. RAG는 검색한 자료를 답 생성의 근거로 제공하는 방식이고요. 이 수업의 에이전트는 목표를 받고 도구의 결과에 따라 다음 행동을 정하되, 완료하거나 멈춤 조건에 이르면 끝냅니다. 도구는 실제 조회, 계산, 제출을 수행하는 기능입니다.

실행 고리는 요청 해석, 계획, 호출, 관찰을 되풀이하다가 종료하는 틀이에요. 신선도는 관측된 값이 판단 시점에서 얼마나 오래됐는지를 나타냅니다. 근거의 상태는 셋이었죠. 근거의 지지를 받음, 확인하지 못함, 지지 근거 없음.

구조화 출력은 정해진 항목과 형식을 갖춘 JSON 결과고요. 실행 기록은 도구 호출의 시각과 입력, 출력, 걸린 시간을 남긴 기록입니다.

[SLIDE 123] 오늘 다룬 핵심 개념과 주요 용어 정리

에이전트 용어에 이어, 스킬과 도구 연결의 용어를 정리하겠습니다. 표에 아홉 용어가 있고, 영문 정식 명칭은 화면에 같이 적어 뒀어요.

위에서부터 갈게요. 에이전트 스킬은 특정한 일을 수행하는 방법을, 이름을 붙여 적어 둔 에이전트의 지침 묶음입니다. MCP는 AI 애플리케이션이 외부 도구와 데이터에 연결하는 방식을 정한 개방형 프로토콜이고요.

호스트는 모델을 가진 애플리케이션이고, 클라이언트는 그 안에서 서버 하나와 연결을 유지합니다. 서버는 시스템의 기능을 내주는 프로그램, 도구는 서버가 내주는 호출할 수 있는 기능이에요. 입력 스키마는 도구에 넘길 값의 이름과 형을 정한 정의입니다.

stdio와 Streamable HTTP는 연결 방식이죠. 하위 프로세스의 표준 입출력으로 주고받거나, HTTP 요청으로 주고받습니다. 읽기 전용 표시는 도구가 시스템을 바꾸지 않는다고 서버가 스스로 다는 표시예요. 최소 권한 원칙은 맡은 일에 필요한 권한만 주는 원칙이고, 마지막으로 자리표시자는 설정에 비밀값 대신 적어 두는 값의 이름과 자리입니다.

[SLIDE 124] Review Quiz: 핵심 흐름 되짚어보기

이름과 뜻을 정리했으니, 그걸 판단에 쓸 수 있는지 퀴즈 여섯 문제로 확인해 보겠습니다. 용어를 외웠는지보다, 상황을 보고 에이전트가 무엇을 해야 하는지 고를 수 있는지가 중요해요. 문제마다 고를 시간을 드릴게요. 여섯 문항을 다 푼 뒤에 정답과 까닭을 같이 보겠습니다.

[SLIDE 125] Quiz 1. 처리 기록의 한 장면

첫 번째 문제입니다. 에이전트가 원인 진단을 불렀어요. 그리고 그 결과에 든 원인과 고장 유형의 식별자를 넣어서 대안 평가를 불렀습니다. 이 장면이 보여 주는 실행 고리의 단계는 뭘까요?

1번, 요청 해석.
2번, 결과 관찰에 이은 다음 호출.
3번, 재시도.
4번, 종료.

[SLIDE 126] Quiz 2. 읽지 못한 값과 묵은 값

두 번째 문제예요. 에이전트가 원인을 가리려고 합니다. 그런데 유온의 최신 값은 3분 전의 것이었고, 냉각 효율 조회는 실패했어요. 이때 에이전트의 올바른 동작은 무엇일까요?

1번, 3분 전의 유온과 판별 기준 70 %를 써서 쿨러 핀 오염으로 제출한다.
2번, 냉각 효율을 0 %로 적고 유온을 가장 많이 내리는 조치를 추천한다.
3번, 원인 후보에 올라 있는 쿨러 핀 오염을 확인된 원인으로 적는다.
4번, 원인을 적지 않고, 확인하지 못한 값과 그 이유를 담아 보류한다.

[SLIDE 127] Quiz 3. 허용되지 않은 도구

세 번째 문제입니다. 에이전트가 일을 끝내려면 허용 목록에 없는 도구가 필요하다고 판단했어요. 유온은 계속 오르고 있고요. 이때 일어나야 하는 일은 무엇일까요?

1번, 허용된 도구를 여럿 조합해 같은 효과를 내는 길을 찾는다.
2번, 급한 상황이므로 한 번만 실행하고 그 사실을 기록에 남긴다.
3번, 실행하지 않고 사람에게 묻는다. 답을 받아도 실행 권한은 그대로다.
4번, 사람에게 물은 뒤 답을 받으면 그 도구를 직접 실행한다.

[SLIDE 128] Quiz 4. 스킬에 적힌 조회가 실행되지 않은 까닭

네 번째 문제입니다. 스킬에는 업무 DB의 생산 오더 조회 도구를 직접 호출하라는 단계가 있어요. 그런데 실행해 보니 그 값을 받지 못했습니다. 이 에이전트에 연결된 서버는 지식 그래프 서버와 판단 엔진 서버, 이 둘뿐이에요. 먼저 고칠 곳으로 가장 알맞은 건 무엇일까요?

1번, 스킬의 절차에 납기 오더를 조회하는 도구의 입력 스키마를 옮겨 적는다.
2번, 에이전트에 업무 DB 서버를 연결한다.
3번, 스킬의 설명을 더 자세히 고쳐 쓴다.
4번, 허용 목록에서 판단 엔진 서버의 도구를 뺀다.

[SLIDE 129] Quiz 5. 시간을 넘긴 재고 조회

다섯 번째 문제입니다. 예비품 재고를 조회하는 호출이, 정해진 시간 안에 끝나지 못했어요. 포털은 시간 초과로 결과를 받지 못했다고 알렸습니다. 결과에 적을 내용으로 가장 알맞은 건 무엇일까요?

1번, 재고가 0개라고 적는다.
2번, 지난 실행에서 본 재고 값을 적는다.
3번, 재고 항목을 결과에서 지운다.
4번, 조회에 실패해 재고를 확인하지 못했다고 적는다.

[SLIDE 130] Quiz 6. 등록한 서버에서 에이전트가 쓸 수 있는 도구

마지막 문제입니다. HYD 포털에 새로 등록한 서버가 도구 세 개를 내줍니다. 설정에 대한 연결 검사는 통과했고, 강사의 별도 읽기 확인은 없는 상황입니다. 행을 읽는 도구는 읽기 전용으로 표시했고, 행을 고치는 도구는 쓰기로 표시했습니다. 내보내기 도구는 표시가 없어요. 연결 검사를 거친 뒤, 에이전트의 허용 목록에 오르는 도구는 무엇일까요?

1번, 행을 읽는 도구 하나.
2번, 행을 읽는 도구와 내보내기 도구.
3번, 세 도구 모두.
4번, 오르는 도구가 없다.

[SLIDE 131] 정답과 판단 기준

여섯 문제를 다 골랐으면, 정답을 확인해 볼게요. 고른 답과 그 까닭을 표와 함께 대조해 보세요. 첫 문항의 정답은 2번입니다. 서로 다른 두 도구를 이어서 불렀고, 앞 호출의 결과가 뒤 호출의 입력이 됐죠. 결과를 관찰한 게 다음 행동을 정한 장면이에요.

3번 재시도와 헷갈렸을 수 있어요. 재시도는 뜻대로 되지 않은 일을 방법을 바꿔서 다시 하는 겁니다. 이 장면에서는 앞 호출이 제대로 끝났고, 그 결과를 가지고 다른 도구를 불렀죠.

둘째 문항은 4번입니다. 묵은 값과 실패한 조회로는 근거가 확인되지 않아요. 1번은 3분 전의 값을 지금의 값처럼, 기준 70을 측정값처럼 썼습니다. 2번은 실패를 0으로 바꿨고, 3번은 후보를 확인된 원인으로 바꿨고요. 셋 다 결과의 모양은 그럴듯하지만, 근거가 없는 자리를 채운 거예요.

셋째 문항은 3번입니다. 허용 목록 밖의 도구는 실행되지 않고, 사람에게 묻는 질문으로 올라가요. 1번은 허용된 도구로 막힌 일을 돌아가는 거고, 2번은 급하다는 이유로 권한을 넘는 겁니다. 4번은 사람에게 물었다는 점에서 그럴듯하죠. 하지만 답변이 실행 권한을 바꾸지는 않아요.

처음에는 AI가 추천안을 냈다는 결과만 보였지만, 호출 기록을 펼치니 역할이 나뉘었죠. 모델은 다음 행동을 정하고, 도구는 실제 값을 읽고 계산하며, 사람은 근거와 권한의 경계를 확인합니다. 이 구분을 지켜야 다른 업무에 에이전트를 적용할 때도 무엇을 맡기고 무엇을 확인할지 정할 수 있습니다.

이어서 스킬과 도구 연결에 관한 나머지 세 문제의 판단 기준을 확인하겠습니다.

[SLIDE 132] 정답과 판단 기준

세 문제를 다 보셨으면, 고른 답을 근거와 함께 맞춰 보겠습니다. 네 번째 문제의 정답은 2번, 에이전트에 업무 DB 서버를 연결하는 겁니다. 스킬에는 방법이 이미 적혀 있었어요. 빠진 건 그 방법을 실행할 연결이죠. 납기 오더를 내주는 도구는 업무 DB 서버에 있으니까, 지정된 조회를 수행하려면 그 서버를 연결하고 호출 권한과 입력을 확인해야 합니다. 1번과 3번은 스킬을 고치는 일이라서 빠진 연결을 채우지 못하고요, 4번은 남아 있던 도구까지 줄여 버립니다.

다섯 번째 문제의 정답은 4번, 조회에 실패해서 재고를 확인하지 못했다고 적는 겁니다. 실패한 조회는 재고가 없다는 걸 알려 주지 않았어요. 재고를 모른다는 것만 알려 줬죠. 1번은 모르는 걸 없는 걸로 바꿨고, 2번은 지난 값을 지금 값처럼 썼고, 3번은 확인하지 못했다는 사실까지 지워 버렸습니다.

여섯 번째 문제의 정답은 1번, 행을 읽는 도구 하나예요. 이 문제의 HYD 기본 판정에서는 읽기 표시가 있고 이름도 쓰기가 아닌 도구만 읽기로 인정합니다. 고치는 도구는 쓰기로 표시돼 있으니 걸러지고, 내보내기 도구는 표시가 없어서 읽는 도구인지 알 수 없으니 걸러져요. 2번을 골랐다면, 표시가 없는 도구를 읽기로 본 겁니다.

여기까지가 이번 회차입니다.

다음 회차에서는 에이전트가 멈추고 사람에게 물어야 하는 순간을 다룹니다. 사람의 승인과 거절이 업무 결과를 어떻게 바꾸는지 확인하고, 요구사항이 바뀌어도 기존 기능을 지키는 방법도 살펴볼 거예요. 수고 많으셨습니다. 감사합니다.